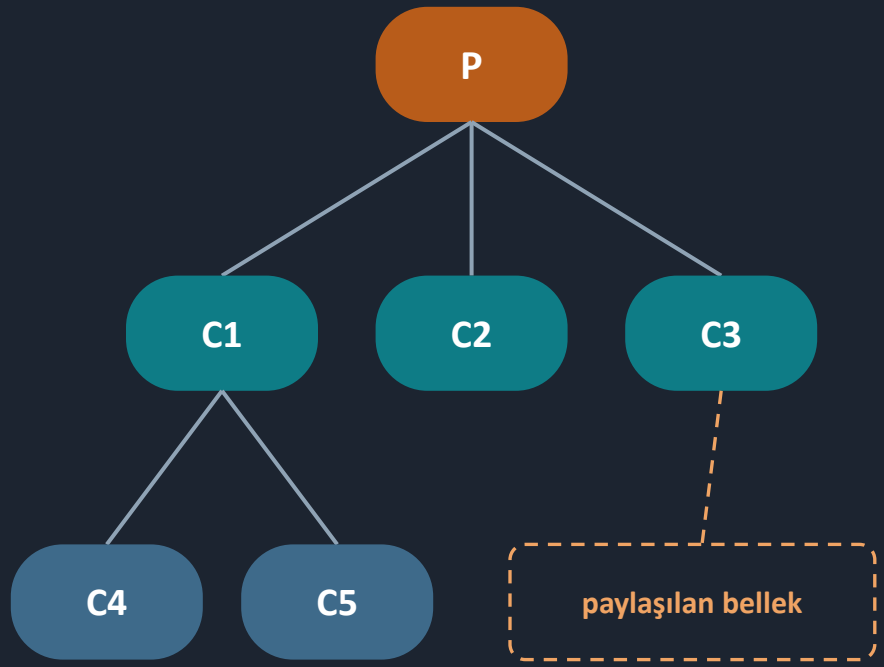
KARADENİZ TEKNİK ÜNİVERSİTESİ

Yazılım Geliştirme Bölümü

İŞLETİM SİSTEMLERİ

Hafta 3

Süreç Kavramı, Süreç İşlemleri ve Süreçler Arası İletişim



Dr. Öğr. Üyesi Sayyad Alizadeh

Bu Haftanın Yol Haritası

1

Süreç kavramı

Program ile süreç arasındaki fark nedir?

2

Süreç durumları

Bir süreç yaşamı boyunca hangi durumlardan geçer?

3

PCB ve bağlam değiştirme

İS bir süreci nasıl kaydeder ve geri yükler?

4

Zamanlama kuyrukları

Süreçler sistemde nerede bekler?

5

Süreç oluşturma

fork(), exec() ve wait() nasıl birlikte çalışır?

6

Süreç sonlandırma

Zombi ve yetim süreç nedir?

7

Süreçler arası iletişim

Paylaşımlı bellek mi, mesaj geçirme mi?

8

IPC örnekleri

POSIX paylaşımlı bellek, borular, soket, RPC

Öğrenme Çıktıları

Bu haftanın sonunda öğrenciler:

1. Program ile süreci ayırt eder ve sürecin bellek düzenini açıklar
2. Süreç durumlarını ve geçişleri tetikleyen olayları durum diyagramıyla açıklar
3. PCB'nin içeriğini ve bağlam değiştirmedeki rolünü açıklar
4. fork() içeren kodlarda oluşan süreç sayısını süreç ağacıyla belirler
5. fork(), exec() ve wait() çağrılarını ayırt eder; zombi ve yetim süreçleri açıklar
6. Paylaşımlı bellek ile mesaj geçirmeyi karşılaştırır ve uygun IPC mekanizmasını seçer

Geçen Haftadan Bu Haftaya

HAFTA 2'DE GÖRDÜKLERİMİZ

Sistem çağrısı ve API

Süreç denetimi çağrıları

fork() · exec() · wait()

Kabuk bir sistem programıdır

- Kabuk bir komutu çalıştırırken fork() ve exec() kullanır
- Ama bu çağrıların oluşturduğu ve yönettiği şey tam olarak nedir?
- Bu hafta: o varlığı — süreci — ayrıntılı inceliyoruz

Hafta 2 konuları yeniden anlatılmaz; sistem çağrıları yalnızca süreç işlemleri için kullanılır.

BU HAFTANIN TEMEL SORUSU

İşletim sistemi bir programı çalıştırdığında aslında neyi yönetiyor?

İpucu: programı değil, programın çalışan örneğini — süreci (process)

Program ile Süreç Aynı Şey Değildir

Program

pasif varlık

- Diskte saklanan komutlar listesi
- Çalıştırılabilir dosya (executable)
- Kendi başına hiçbir şey yapmaz

`Örn. /usr/bin/editor`

belleğe yüklenip çalıştırılınca →

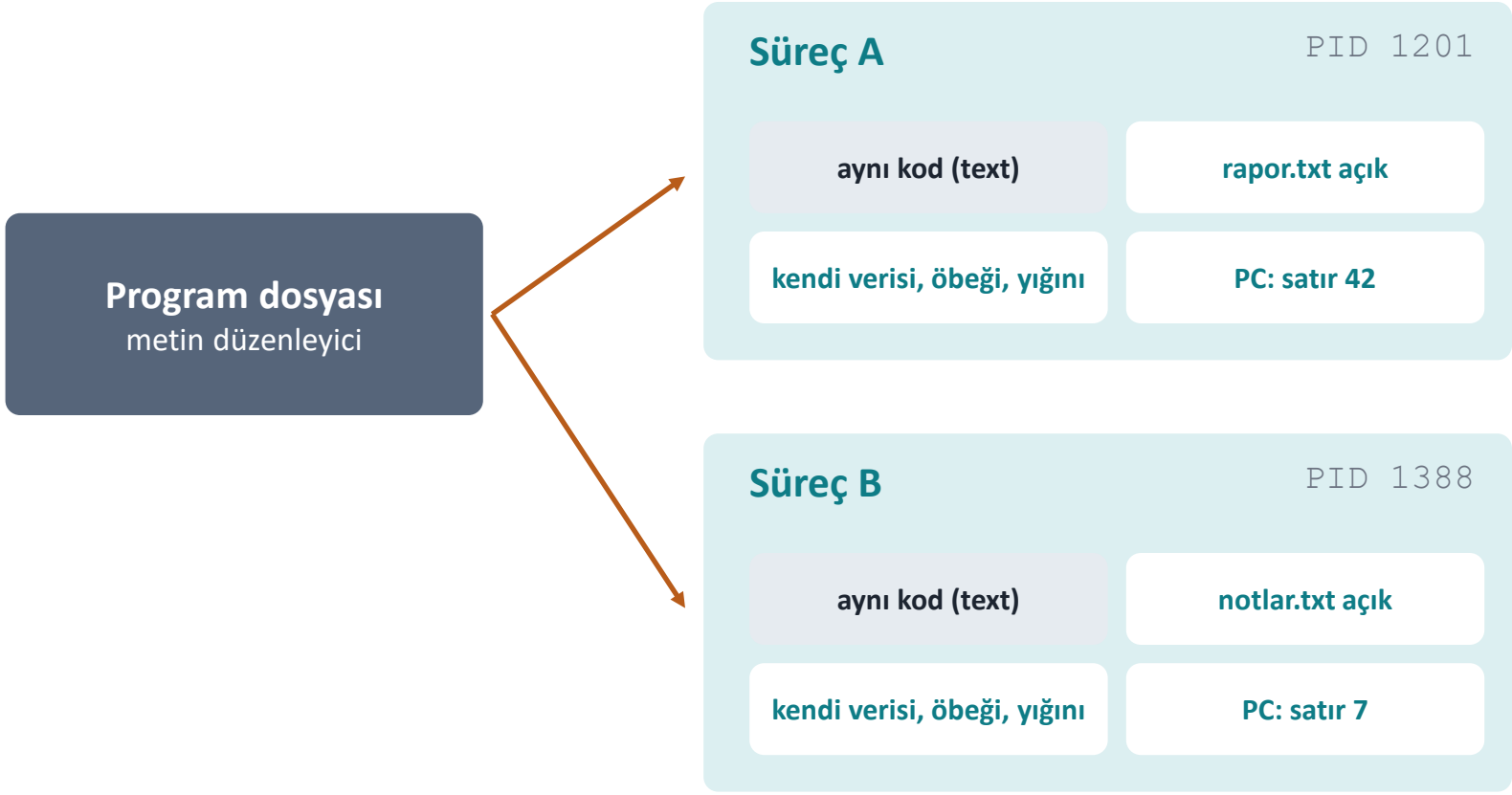
Süreç (Process)

etkin varlık: çalışmakta olan program

- **Program sayacı (PC)**
- **CPU yazmaçları**
- **Bellek: kod, veri, öbek, yığın**
- **Açık dosyalar ve diğer kaynaklar**
- **O anki durum**

Program yürütülebilir dosya belleğe yüklendiğinde sürece dönüşür; süreç, İS'deki iş birimidir.

Aynı Programdan Birden Çok Süreç

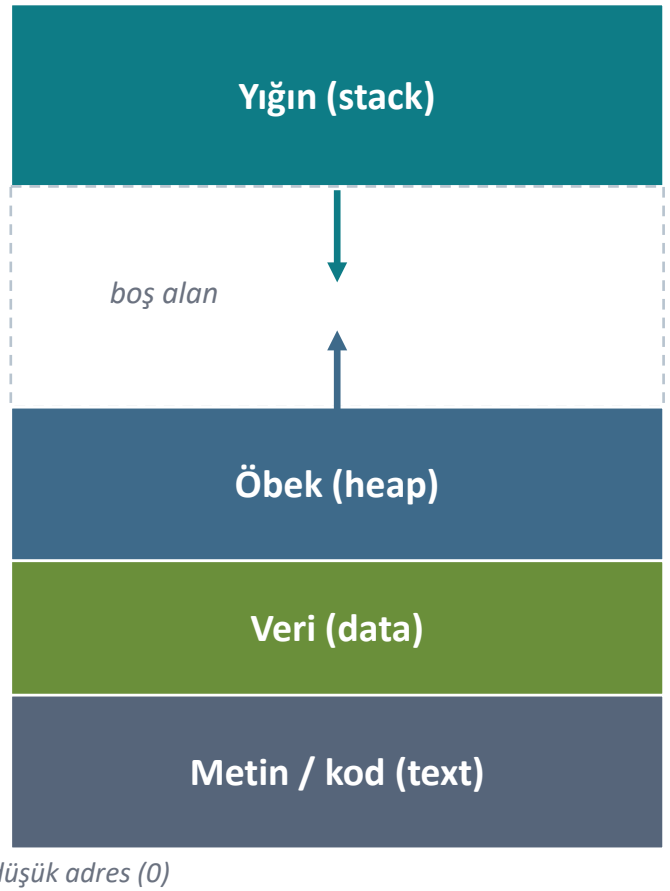


Örnek: aynı metin düzenleyiciyi iki kez açmak ya da birden çok kullanıcının aynı tarayıcıyı çalıştırması

- Kod bölümü aynıdır
- Veri, öbek ve yığın bölümleri farklıdır
- Her biri ayrı bir yürütme dizisidir

PID değerleri temsilîdir. Kaynak: Silberschatz vd., Bölüm 3.1.1

Bir Sürecin Bellekteki Düzeni



Silberschatz vd., Şekil 3.1 temel alınmıştır

Metin (kod)
Programın makine komutları

Veri
Global ve statik değişkenler

Öbek (heap)
Çalışma sırasında dinamik ayrılan bellek (ör. malloc); yukarı doğru büyür

Yığın (stack)
Geçici veriler: işlev parametreleri, dönüş adresleri, yerel değişkenler; aşağı doğru büyür

Not: Bu şema kavramsaldır; gerçek adres uzayı düzeni sisteme ve mimariye göre daha karmaşık olabilir.

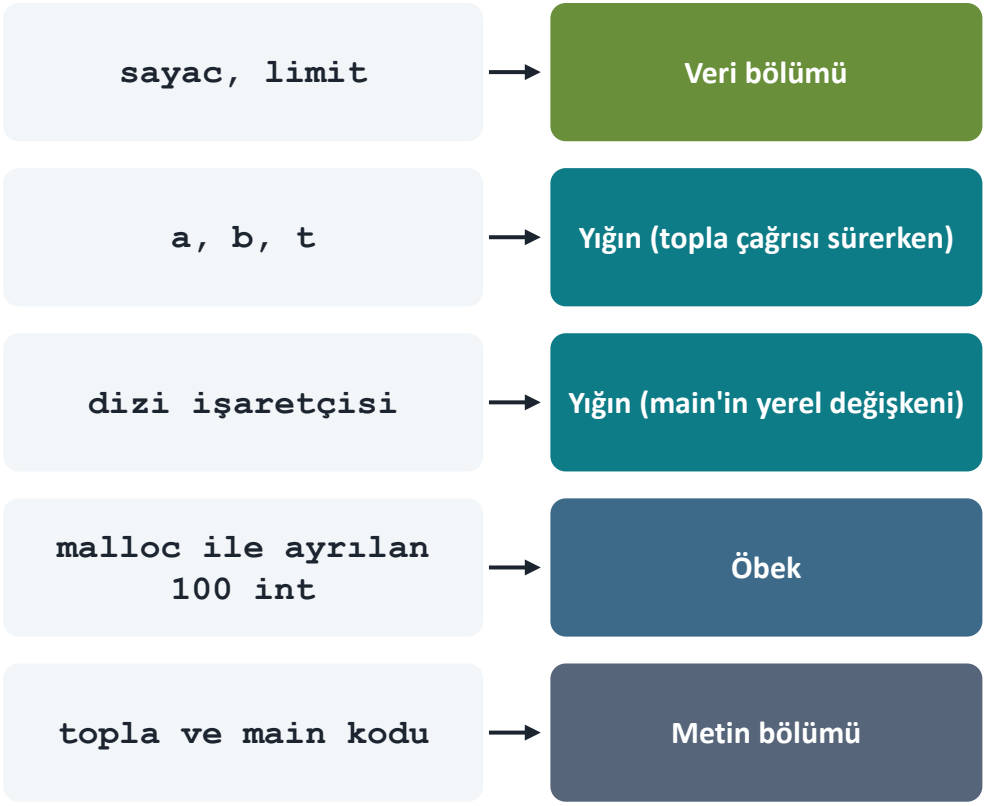
Uygulama: Hangi Değişken Nerede Yaşar?

```c
int sayac = 0;              // global
static int limit = 10;      // statik

int topla(int a, int b) {
    int t = a + b;          // yerel
    return t;
}

int main(void) {
    int *dizi = malloc(100 * sizeof(int));
    sayac = topla(2, 3);
    return 0;
}
```

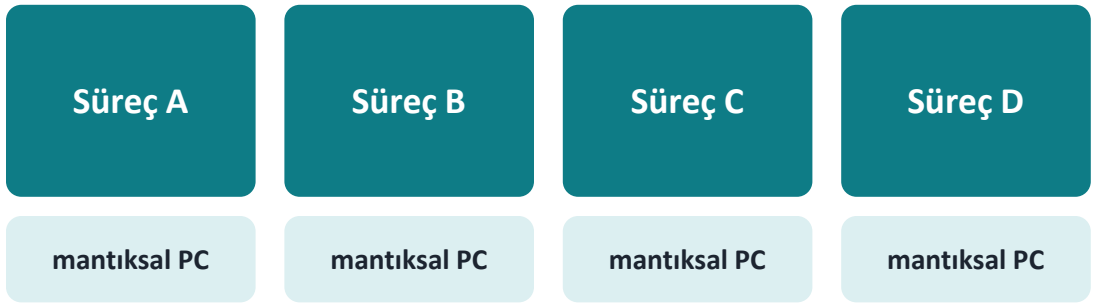
ÇÖZÜM



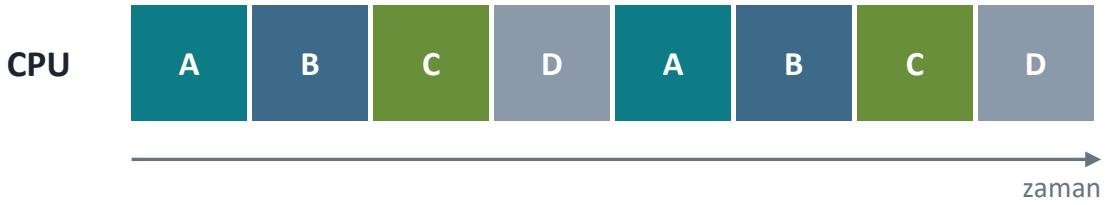
Dikkat: dizi işaretçisi yığında, işaret ettiği veri öbektedir.

Süreç Modeli: Mantıksal ve Fiziksel Program Sayacı

Kavramsal bakış: her sürecin kendi sanal CPU'su



Gerçekte: tek CPU, tek fiziksel PC — CPU süreçler arasında hızla geçiş yapar



Sonuç

- Herhangi bir anda tek çekirdekte yalnızca bir süreç gerçekten çalışır
- Süreç dışarıdan bakınca kesintisiz ilerliyor gibi görünür; geçişlerde durumu kaydedilip geri yüklenir (birazdan: PCB ve bağlam değiştirme)
- Süreç yürütmesi, zamanlamaya bağlı olarak düzgün hızda olmayabilir

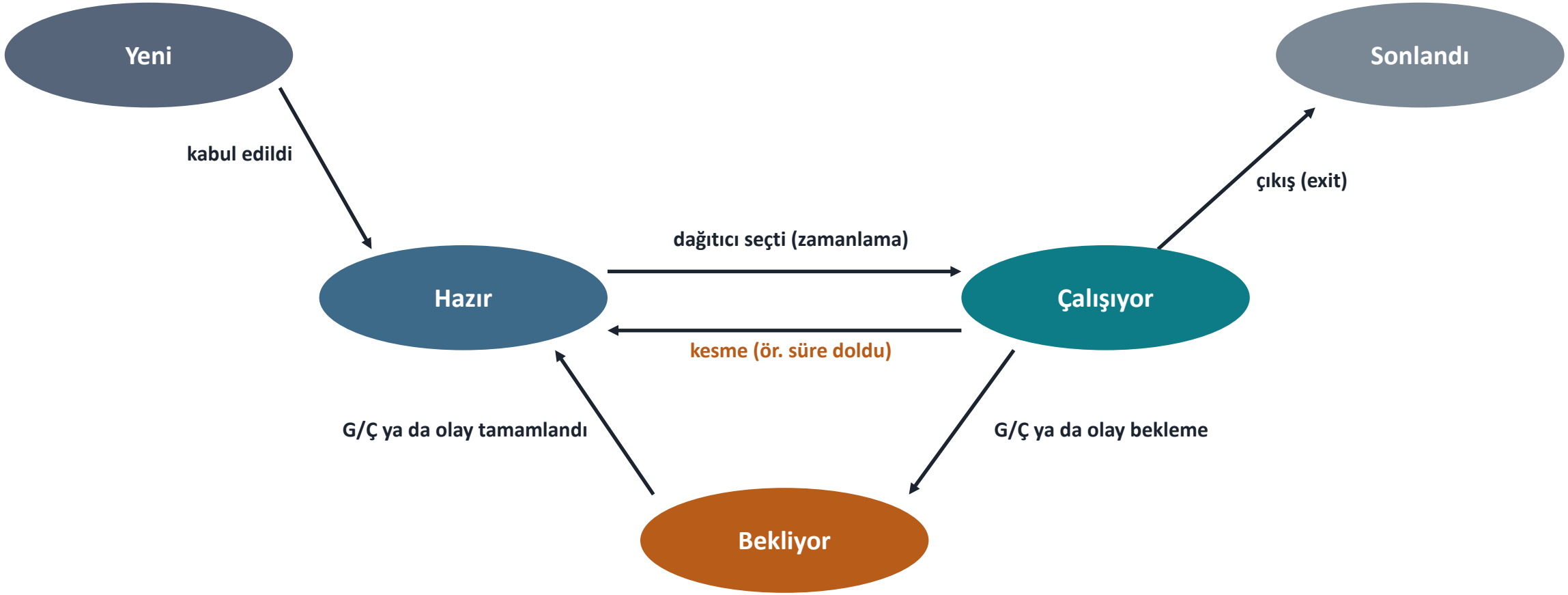
Tanenbaum & Bos, Bölüm 2.1.1 (Şekil 2-1) temel alınmıştır

Süreç Durumları (Process States)

Yeni (New)	Süreç oluşturuluyor
Hazır (Ready)	CPU'ya atanmayı bekliyor
Çalışıyor (Running)	Komutları CPU'da yürütülüyor
Bekliyor (Waiting / Blocked)	Bir olayın gerçekleşmesini bekliyor (ör. G/Ç tamamlanması)
Sonlandı (Terminated)	Yürütmesi bitti

Önemli: Tek bir işlemci çekirdeğinde herhangi bir anda yalnızca bir süreç Çalışıyor durumunda olabilir; birçok süreç Hazır ya da Bekliyor durumunda olabilir.

Süreç Durum Diyagramı



Bekliyor → Çalışıyor doğrudan geçişi yoktur: olay tamamlanan süreç önce Hazır olur.

Silberschatz vd., Şekil 3.2

Geçişleri Tetikleyen Olaylar

Geçiş	Ne zaman olur?	Örnek
Yeni → Hazır	İS süreci kabul eder, kaynaklarını hazırlar	Kullanıcı bir programı başlattı
Hazır → Çalışıyor	CPU Zamanlayıcısı süreci seçer, dağıtıcı CPU'yu verir	Sıra sürece geldi
Çalışıyor → Bekliyor	Süreç bir olayı beklemek zorundadır	Diskten okuma isteği
Bekliyor → Hazır	Beklenen olay gerçekleşir	Disk okuması bitti (kesme)
Çalışıyor → Hazır	Süreç CPU'yu kesme sonucu bırakır (kesintili zamanlama)	Donanım Zamanlayıcısı kesmesi
Çalışıyor → Sonlandı	Süreç biter ya da sonlandırılır	exit() çağrısı

Ayrım: Donanım Zamanlayıcısı (Timer) kesmeyi üreten donanımdır; CPU Zamanlayıcısı (CPU Scheduler) sıradaki süreci seçen yazılım kararıdır. Seçim algoritmaları Hafta 5'te.

Senaryo: Bir Sürecin Yaşam Öyküsü



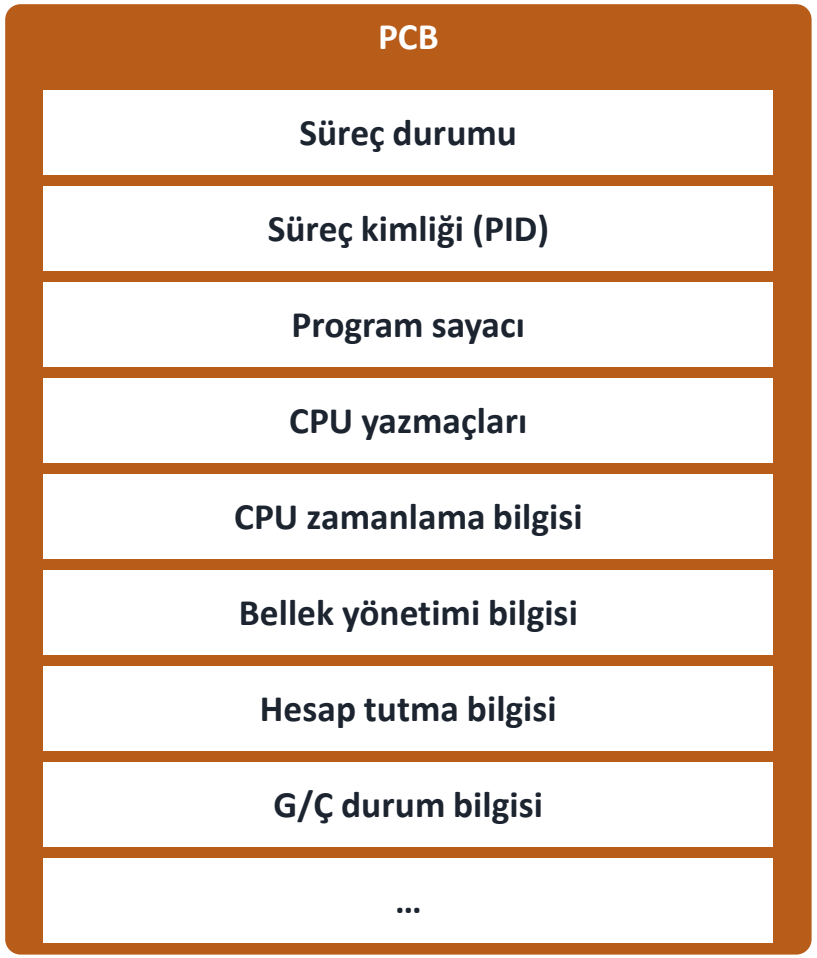
Gözlem

Süreç yaşamı boyunca Hazır ve Çalışıyor durumları arasında birçok kez gidip gelebilir. Bekleyen süreç CPU kullanmaz.

Düşünün

4. adımda süreç beklerken CPU ne yapar? (İpucu: Hafta 1 — çoklu programlama)

Süreç Kontrol Bloğu (Process Control Block, PCB)



Silberschatz vd., Bölüm 3.1.3 ve Şekil 3.3

Tanım: İşletim sisteminin her süreç için çekirdekte tuttuğu ve o sürece ait tüm bilgileri barındıran veri yapısıdır (görev denetim bloğu da denir).

- Her sürecin bir PCB'si vardır
- PCB'ler çekirdek belleğinde tutulur; süreç onları doğrudan değiştiremez
- PCB, sürecin yönetimi ve daha sonra yürütülmesine devam edilebilmesi için gerekli CPU bağlamı ve yönetim bilgilerini tutar
- Zamanlama kuyrukları aslında PCB'lerin listeleridir

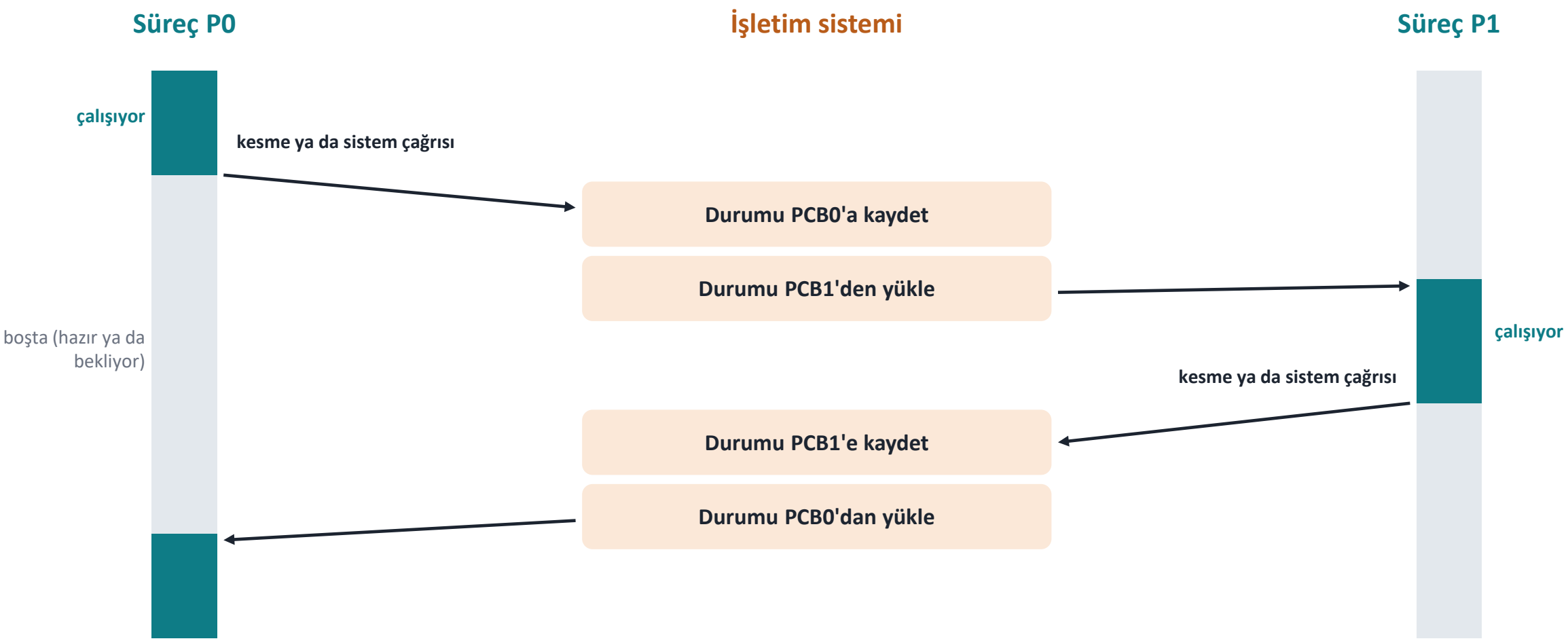
Benzetme: PCB, işletim sisteminin süreç hakkındaki kayıt dosyası gibidir — ama teknik olarak bir çekirdek veri yapısıdır.

PCB Alanları Ne İşe Yarar?

Alan	İçerik	Kim kullanır?
Süreç durumu	Yeni, hazır, çalışıyor, bekliyor…	Zamanlama ve durum geçişleri
Süreç kimliği (PID)	Sürecin benzersiz numarası	Tüm süreç işlemleri
Program sayacı	Yürütülecek bir sonraki komutun adresi	Bağlam değiştirme
CPU yazmaçları	Akümülatör, indeks, yığın işaretçisi, genel amaçlı yazmaçlar…	Bağlam değiştirme
CPU zamanlama bilgisi	Öncelik, kuyruk işaretçileri, diğer parametreler	CPU Zamanlayıcısı (Hafta 5)
Bellek yönetimi bilgisi	Taban/sınır yazmaçları, sayfa ya da bölüt tabloları	Bellek yönetimi (Hafta 9)
Hesap tutma bilgisi	Kullanılan CPU süresi, zaman sınırları, süreç numaraları	Hesap tutma hizmeti
G/Ç durum bilgisi	Ayrılmış G/Ç aygıtları, açık dosyalar listesi	G/Ç ve dosya yönetimi

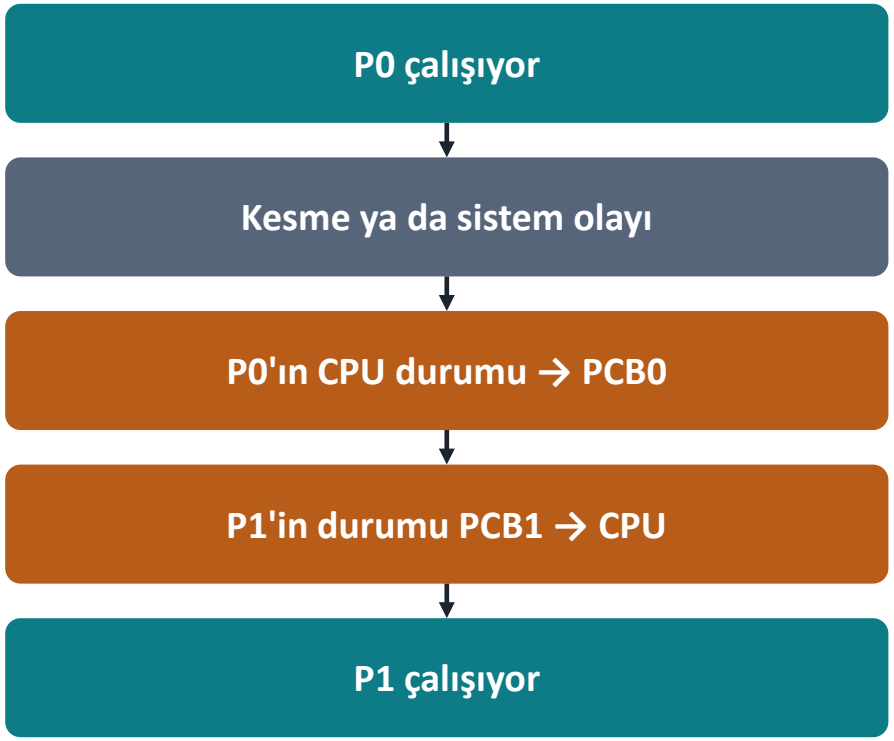
Kaynak: Silberschatz vd., Bölüm 3.1.3

Bağlam Değiştirme (Context Switch): İki Süreç Arasında



Kavramsal zaman çizelgesi (yukarıdan aşağı) · Silberschatz vd., Şekil 3.4

Bağlam Değiştirme Adımları ve Maliyeti



Bağlam (context) nedir?

Sürecin PCB'de temsil edilen durumu: CPU yazmaçlarının değerleri, süreç durumu ve bellek yönetimi bilgisi.

Maliyet: saf ek yük (overhead)

Geçiş sırasında sistem kullanıcı adına yararlı iş yapmaz. Süre sabit değildir; donanıma (ör. yazmaç sayısı, birden çok yazmaç kümesi desteği) ve İS'ye bağlıdır.

Bağlam değiştirme sık olursa CPU zamanının daha büyük kısmı ek yüke gider → zamanlama tasarımında önemli bir ödünleşim (Hafta 5).

Gerçek Sistem Örneği: Linux'ta Süreç Gösterimi

```
struct task_struct {
    long state;              /* süreç durumu */
    pid_t pid;               /* süreç kimliği */
    struct task_struct *parent;  /* ebeveyn */
    struct list_head children;   /* çocuklar */
    struct files_struct *files;  /* açık dosyalar */
    struct mm_struct *mm;    /* adres uzayı */
    /* ... zamanlama bilgisi ve diğerleri */
};
```

- Linux'ta PCB, C yapısı task_struct ile temsil edilir
- Etkin süreçler task_struct'lardan oluşan çift bağlı bir listede tutulur
- Çekirdek, o anda çalışan sürecin yapısına bir işaretçiyle (current) erişir

Kitap temelli örnek: alanlar seçilmiş ve sadeleştirilmiştir; gerçek yapı çok daha büyüktür.

Kaynak: Silberschatz vd., Bölüm 3.1.3 (Linux'ta süreç gösterimi)

Süreçler Nerede Bekler? Zamanlama Kuyrukları

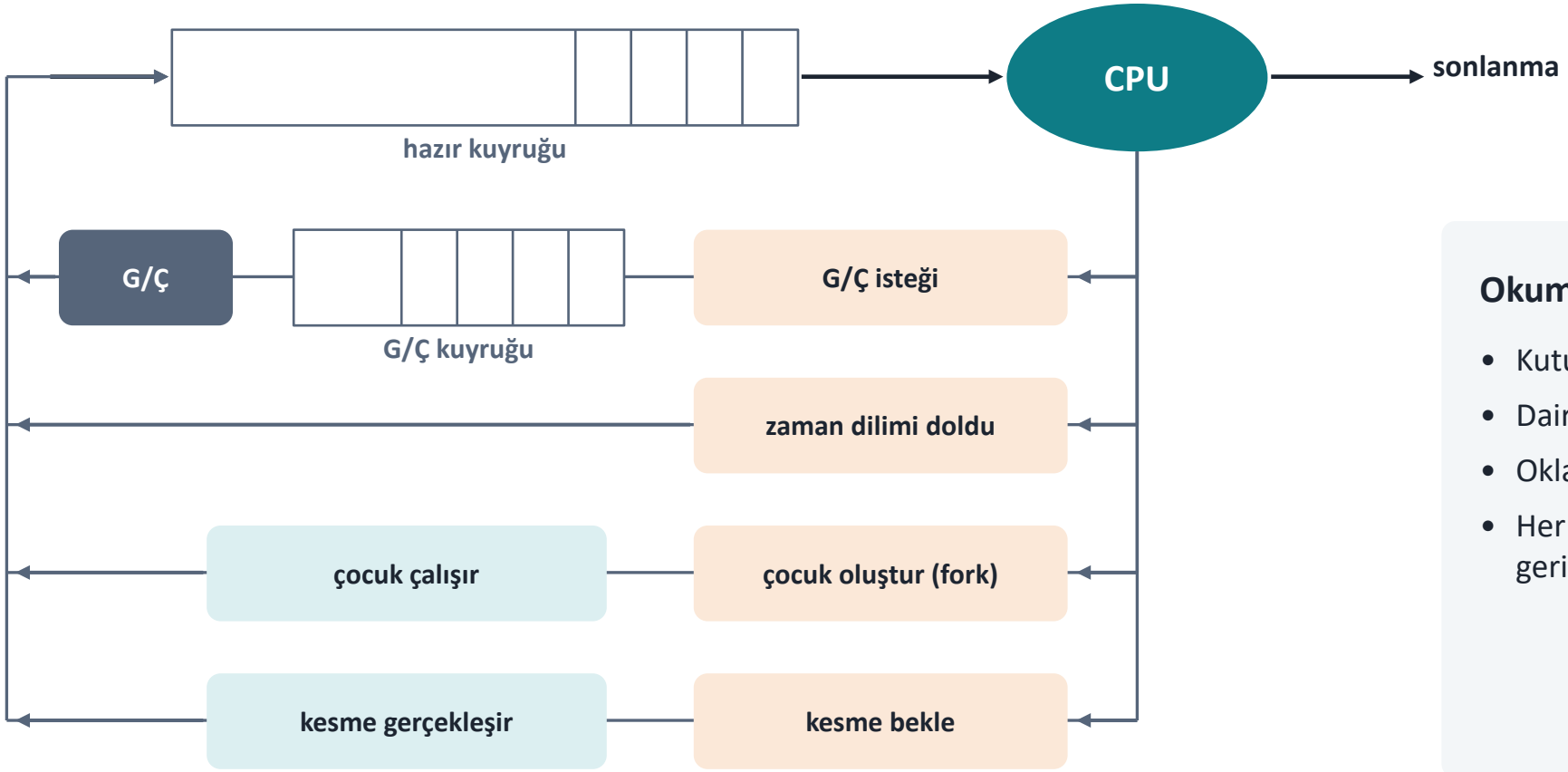
İş kuyruğu (job queue)	Sistemdeki tüm süreçler
Hazır kuyruğu (ready queue)	Ana bellekte, çalışmaya hazır, CPU bekleyen süreçler
Aygıt kuyrukları (device queues)	Belirli bir G/Ç aygıtını bekleyen süreçler; her aygıtın kendi kuyruğu

Hazır kuyruğu: PCB'lerin bağlı listesi



Kaynak: Silberschatz vd., Bölüm 3.2.1 ve Şekil 3.5

Kuyruk Diyagramı: Süreç Sistemde Nasıl Dolaşır?



Okuma

- Kutular: kuyruklar
- Daire: hizmet veren kaynak
- Oklar: süreçlerin akışı
- Her yol sonunda süreci hazır kuyruğuna geri getirir

Silberschatz vd., Şekil 3.6 temel alınarak sadeleştirilmiştir

Zamanlama Birimlerine (Schedulers) Ön Bakış

Uzun vadeli zamanlama birimi
Long-Term Scheduler / iş zamanlayıcısı

- Diskte bekleyen işlerden hangilerinin belleğe alınacağını seçer
- Çoklu programlama derecesini (bellekteki süreç sayısını) denetler
- Seyrek çalışır

CPU Zamanlayıcısı
Short-Term / CPU Scheduler

- Hazır süreçlerden hangisinin CPU'yu alacağını seçer
- Çok sık çalışır; bu yüzden hızlı olmalıdır
- Algoritmalar: Hafta 5

Orta vadeli zamanlama birimi
Medium-Term Scheduler

- Bir süreci geçici olarak bellekten çıkarıp (takas) daha sonra geri alabilir
- Çoklu programlama derecesini düşürmek için kullanılır
- Takas ayrıntısı: Hafta 9

Karıştırmayın: CPU Zamanlayıcısı bir yazılım kararıdır; Donanım Zamanlayıcısı (Timer) ise kesme üreten donanımdır. Kesme, İS'ye denetimi verir; İS de CPU Zamanlayıcısını çalıştırabilir.

Kaynak: Silberschatz vd., Bölüm 3.2.2 · Bu hafta yalnızca görevler tanıtılır; algoritmalar Hafta 5'te

Süreç Karışımı ve Takas: Uzun ve Orta Vadeli Kararlar

G/Ç ağırlıklı süreç (I/O-bound)

Zamanının çoğunu G/Ç yaparak geçirir; kısa CPU kullanımları vardır.

CPU ağırlıklı süreç (CPU-bound)

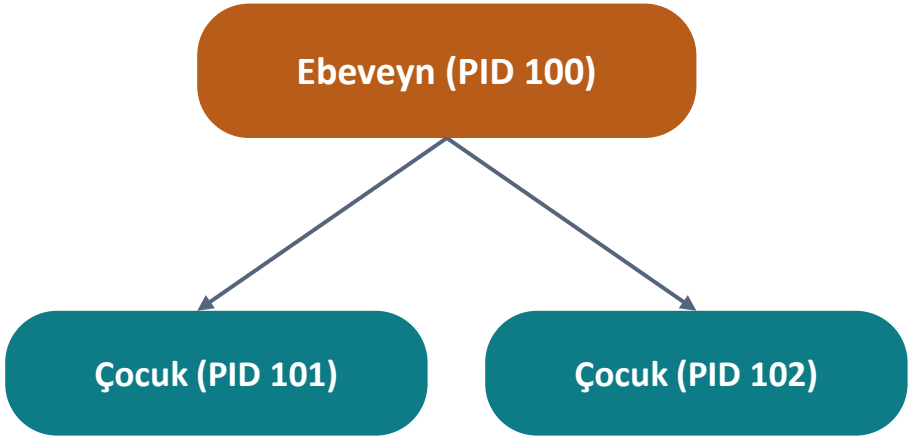
Zamanının çoğunu hesaplama yaparak geçirir; seyrek G/Ç isteği yapar.

Uzun vadeli zamanlama birimi iyi bir karışım seçmelidir: hepsi G/Ç ağırlıklıysa hazır kuyruğu boşalır; hepsi CPU ağırlıklıysa aygıt kuyrukları boş kalır.



Orta vadeli zamanlama (takas) akışı · Kaynak: Silberschatz vd., Bölüm 3.2.2 ve Şekil 3.7

Ebeveyn Süreç ve Çocuk Süreç



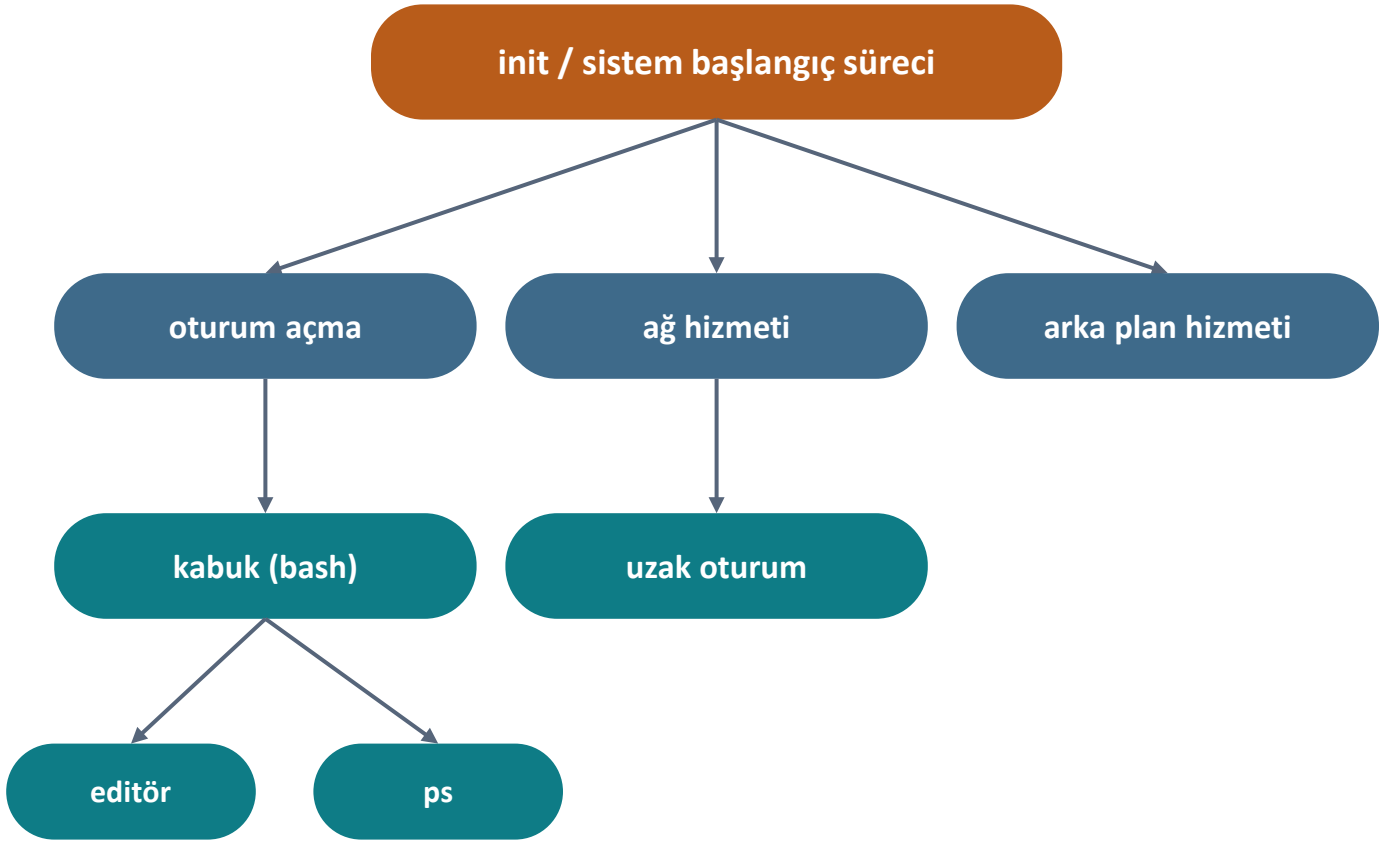
Her süreç benzersiz bir süreç kimliği (PID) ile tanınır. Çocuklar da yeni süreçler oluşturabilir → süreç ağacı

Tasarım sorusu	Olası seçenekler
Kaynaklar	Çocuk kaynaklarını doğrudan İS'den alır ya da ebeveyninin kaynaklarının bir alt kümesiyle sınırlanır
Yürütme	Ebeveyn çocuklarıyla eşzamanlı çalışmayı sürdürür ya da (bazı / tüm) çocukları bitene kadar bekler
Adres uzayı	Çocuk, ebeveynin kopyasıdır ya da çocuğa yeni bir program yüklenir

UNIX'te: fork() ebeveynin kopyası olan çocuğu oluşturur; istenirse exec() çocuğa yeni program yükler.

Kaynak: Silberschatz vd., Bölüm 3.3.1; Tanenbaum & Bos, Bölüm 2.1.2

Süreç Ağacı (Process Hierarchy)



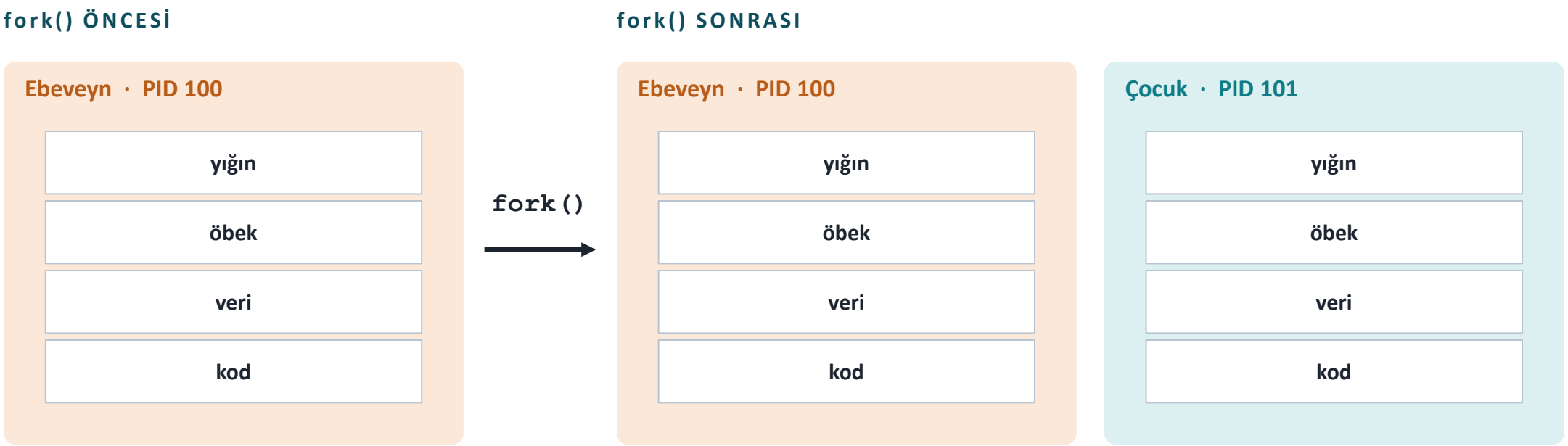
- UNIX/Linux'ta tüm süreçler tek bir kök süreçten türeyen bir ağaç oluşturur
- Her sürecin bir ebeveyni (PPID) vardır
- Kabuktan başlattığınız komutlar kabuğun çocuklarıdır
- Windows'ta süreç hiyerarşisi kavramı yoktur; ebeveyn yalnızca çocuğa erişim için bir tanıtıcı (handle) alır (Tanenbaum)

```
$ ps -el      # süreçleri listele
$ pstree      # süreç ağacını göster
```

Kavramsal örnek: süreç adları temsilîdir

Kaynaklar: Silberschatz vd., Şekil 3.8; Tanenbaum & Bos, Bölüm 2.1.4

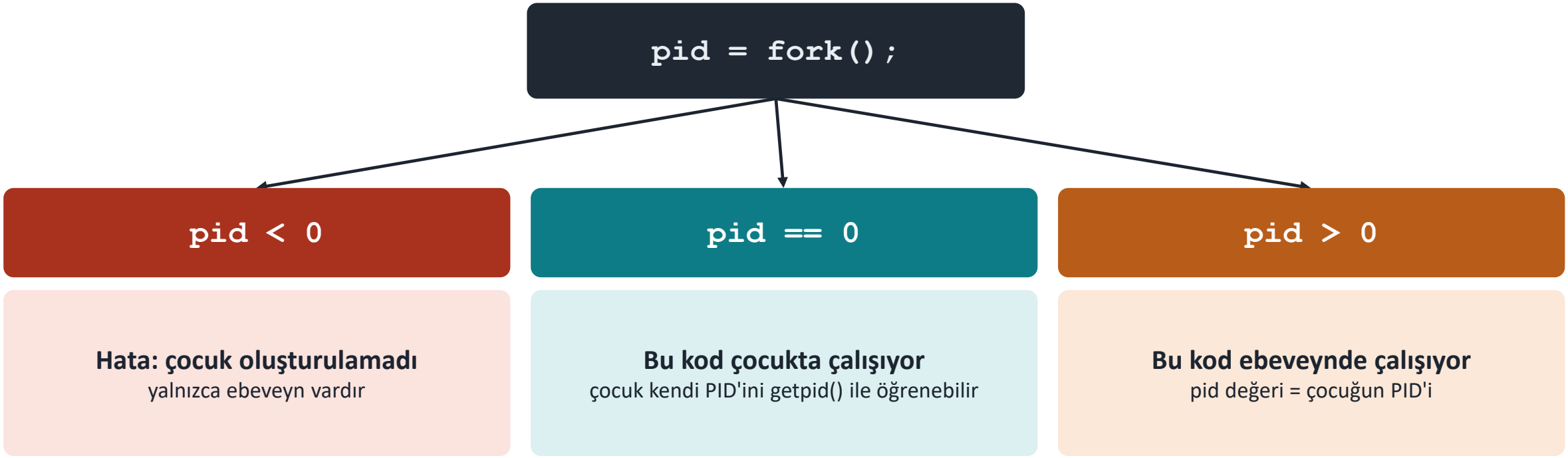
fork(): Yeni Bir Süreç Oluşturmak



- Çocuk, ebeveynin adres uzayının bir kopyasıyla başlar
- Her iki süreç de fork()'tan sonraki komuttan yürütmeye devam eder
- Her iki süreç fork()'tan sonraki aynı kod noktasından devam eder. fork()'un dönüş değeri, ebeveyn ile çocuğu ayırt etmenin temel yoludur; PID'leri farklıdır

Kaynak: Silberschatz vd., Bölüm 3.3.1 · Kopyanın verimli gerçekleştirilmesi (yazarken kopyalama) Hafta 10'da

fork() Dönüş Değeri: Kim Olduğunu Nasıl Anlarsın?



Başarılı bir fork() çağrısı bir kez yapılır ama iki kez döner: bir kez ebeveynde, bir kez çocukta.

Kaynak: Silberschatz vd., Bölüm 3.3.1; Tanenbaum & Bos, Bölüm 1.6.2

Kod: fork() Dönüş Değerini Kullanmak

```c
#include <stdio.h>
#include <unistd.h>

int main(void) {
    pid_t pid = fork();
    if (pid < 0)
        printf("fork hatası\n");
    else if (pid == 0)
        printf("Çocuk: PID=%d\n", getpid());
    else
        printf("Ebeveyn: çocuk PID=%d\n", pid);
    return 0;
}
```

PID değerleri temsilîdir.

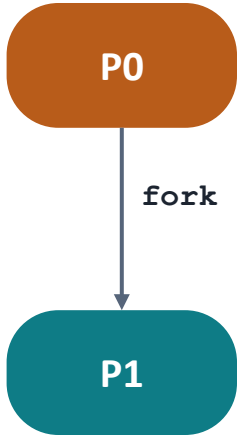
Olası çıktı

```
Ebeveyn: çocuk PID=2814
Çocuk: PID=2814
```

- İki satır da yazdırılır: biri ebeveyn, biri çocuk tarafından
- Ebeveynin gördüğü pid ile çocuğun getpid() değeri aynıdır
- Satırların sırası garanti değildir: hangisinin önce çalışacağına CPU Zamanlayıcısı karar verir

Çözümlü Örnek 1: Tek fork()

```
int main(void) {
    fork();
    printf("Merhaba\n");
    return 0;
}
```



ADIM ADIM

- P0 başlar ve fork()'u çağırır
- P0 bir çocuk oluşturur: P1
- P0 ve P1, printf satırından devam eder

Toplam süreç: 2
“Merhaba” 2 kez yazdırılır

Çözümlü Örnek 2: fork(); fork(); — Adım Adım

```
fork();    /* 1. fork */
fork();    /* 2. fork */
printf("A\n");
```

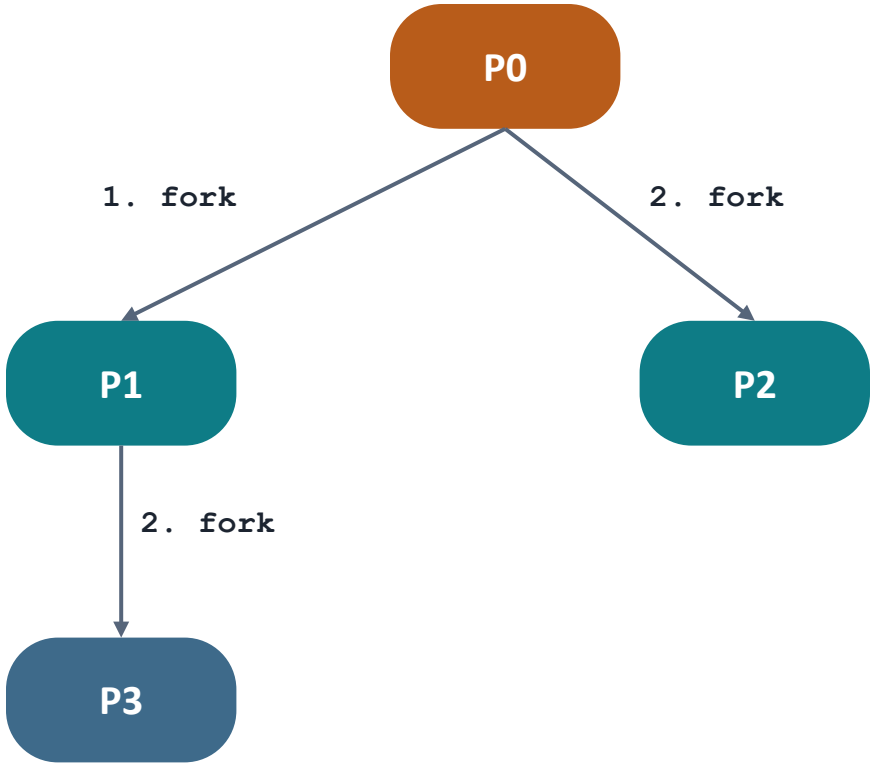
Başlangıç	Yalnızca P0 vardır	**1 süreç**
1. fork	P0 → P1 oluşturur. Artık P0 ve P1 vardır; ikisi de 2. fork'a gelir	**2 süreç**
2. fork	P0 → P2, P1 → P3 oluşturur. Her mevcut süreç bir çocuk üretir	**4 süreç**
printf	4 sürecin her biri “A” yazdırır	**4 satır çıktı**

Her fork, o anda var olan süreç sayısını ikiye katlar

başlangıç	**1**
1. fork sonrası	**2**
2. fork sonrası	**4**

Ağacın çizimi bir sonraki slaytta

Çözümlü Örnek 2: Süreç Ağacı



Ağacı okuma

- P0'ın iki çocuğu vardır: P1 (1. fork) ve P2 (2. fork)
- P1, ebeveyninin kopyası olduğu için 2. fork'u da yürütür → P3
- P2 ve P3 hiç fork yürütmez: oluştuklarında son fork geçilmiştir

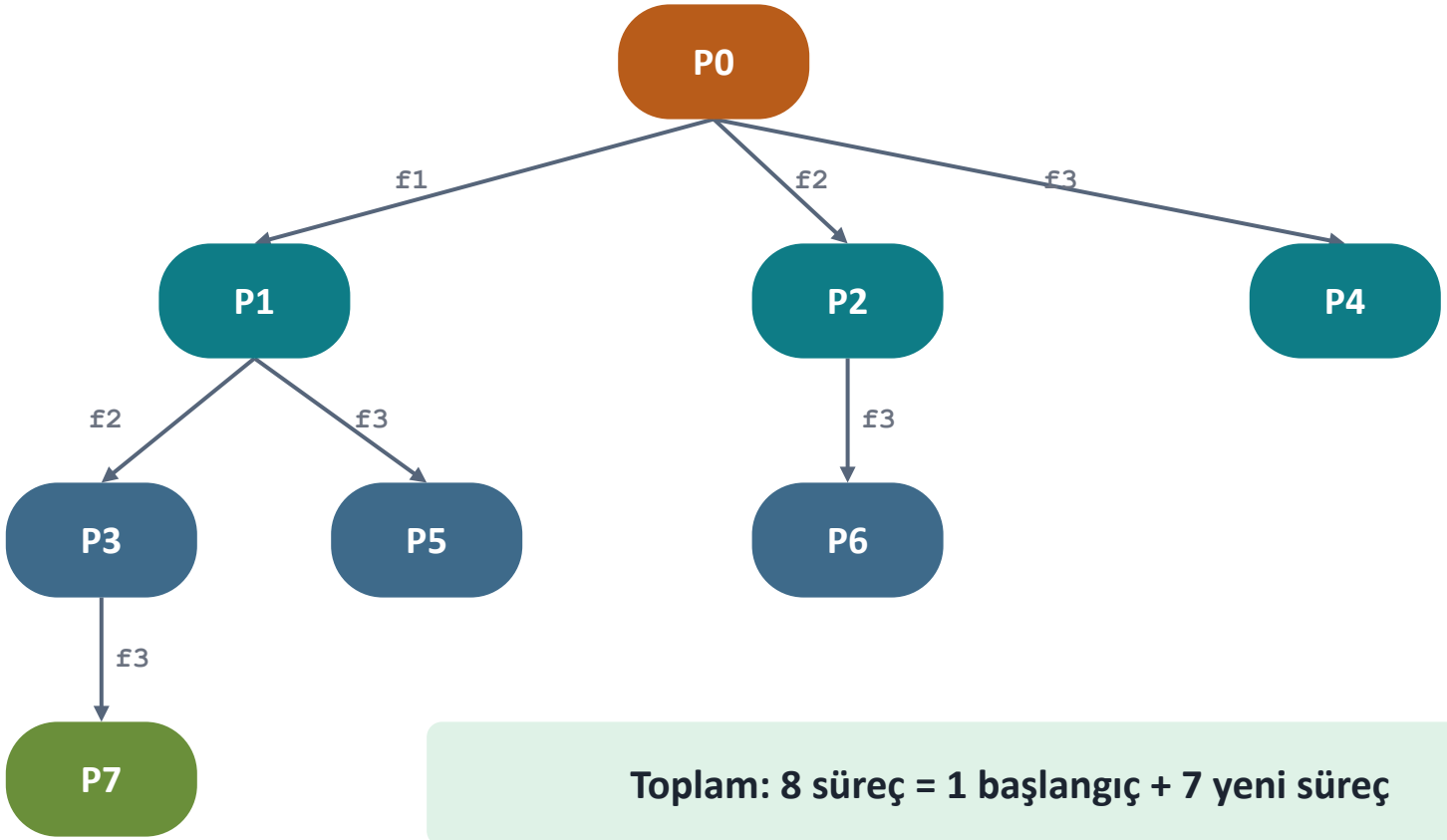
Toplam: 4 süreç · 3 fork() çağrısı başarılı · 4 kez “A”

Not: 3 başarılı fork() → 3 yeni süreç + başlangıç süreci = 4

Çözümlü Örnek 3: Üç fork() Çağrısı

```
fork();   /* f1 */
fork();   /* f2 */
fork();   /* f3 */
```

Adım	Süreç sayısı
Başlangıç	1
f1 sonrası	2
f2 sonrası	4
f3 sonrası	**8**

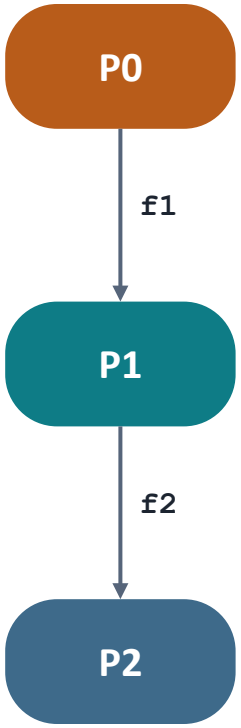


Toplam: 8 süreç = 1 başlangıç + 7 yeni süreç

Kenar etiketleri, çocuğun hangi fork çağrısıyla oluştuğunu gösterir. Süreç numaraları oluşturulma sırasına göre verilmiştir.

Çözümlü Örnek 4: Süreç Sayısı Her Zaman İkiye mi Katlanır?

```
pid_t pid = fork();      /* f1 */
if (pid == 0) {
    fork();              /* f2 */
}
printf("B\n");
```



Kural ezberlemeyin: hangi sürecin hangi fork'u yürüttüğünü izleyin.

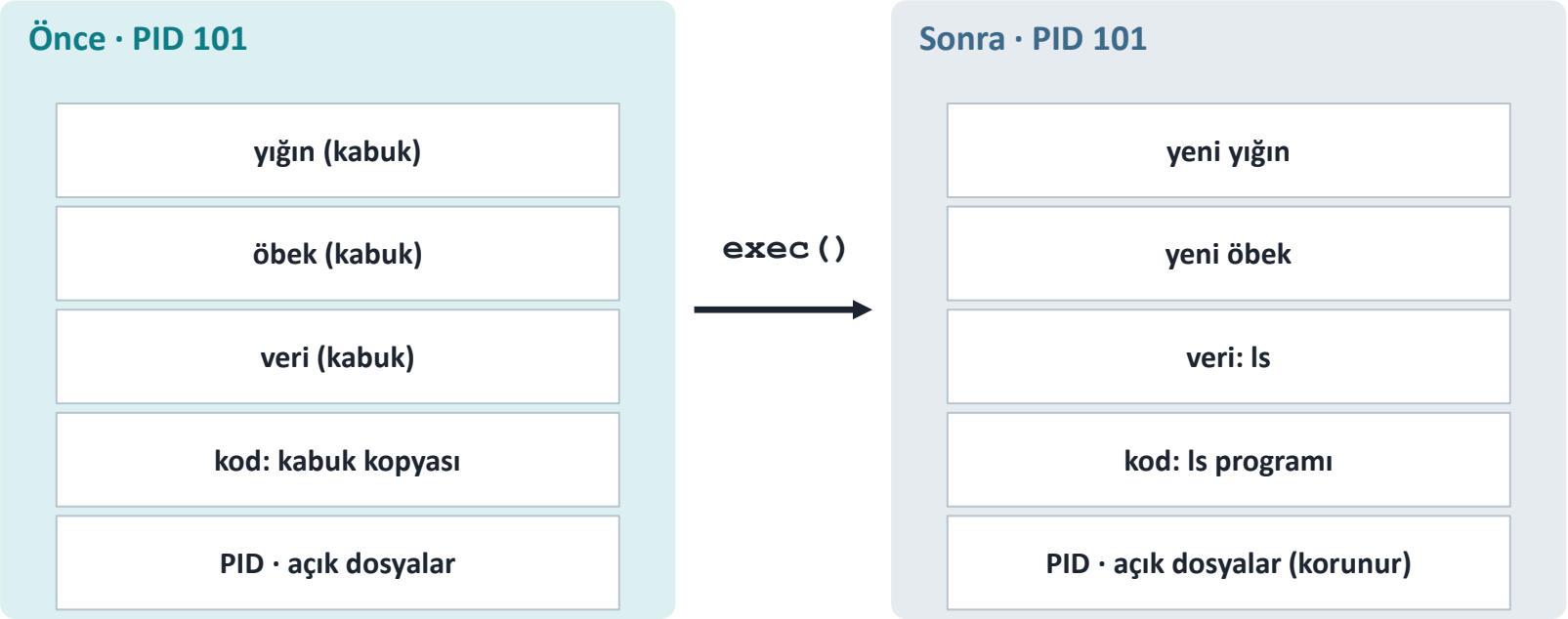
ADIM ADIM

- f1: P0, P1'i oluşturur
- P0'da pid > 0 → if bloğuna girmez
- P1'de pid == 0 → f2'yi yürütür ve P2'yi oluşturur
- P0, P1 ve P2 printf'i yürütür

Toplam: 3 süreç

"B" 3 kez yazdırılır — 4 değil!

exec(): Süreç Görüntüsünü Değiştirmek



- exec() yeni süreç oluşturmaz
- Çağıran sürecin bellek görüntüsü yeni programla değiştirilir
- PID aynı kalır; aynı süreç artık başka bir program çalıştırır
- Başarılı exec() çağıran koda geri dönmez; yalnızca hata olursa döner

Önemli: exec() → aynı süreç, yeni program · fork() → yeni süreç, aynı program

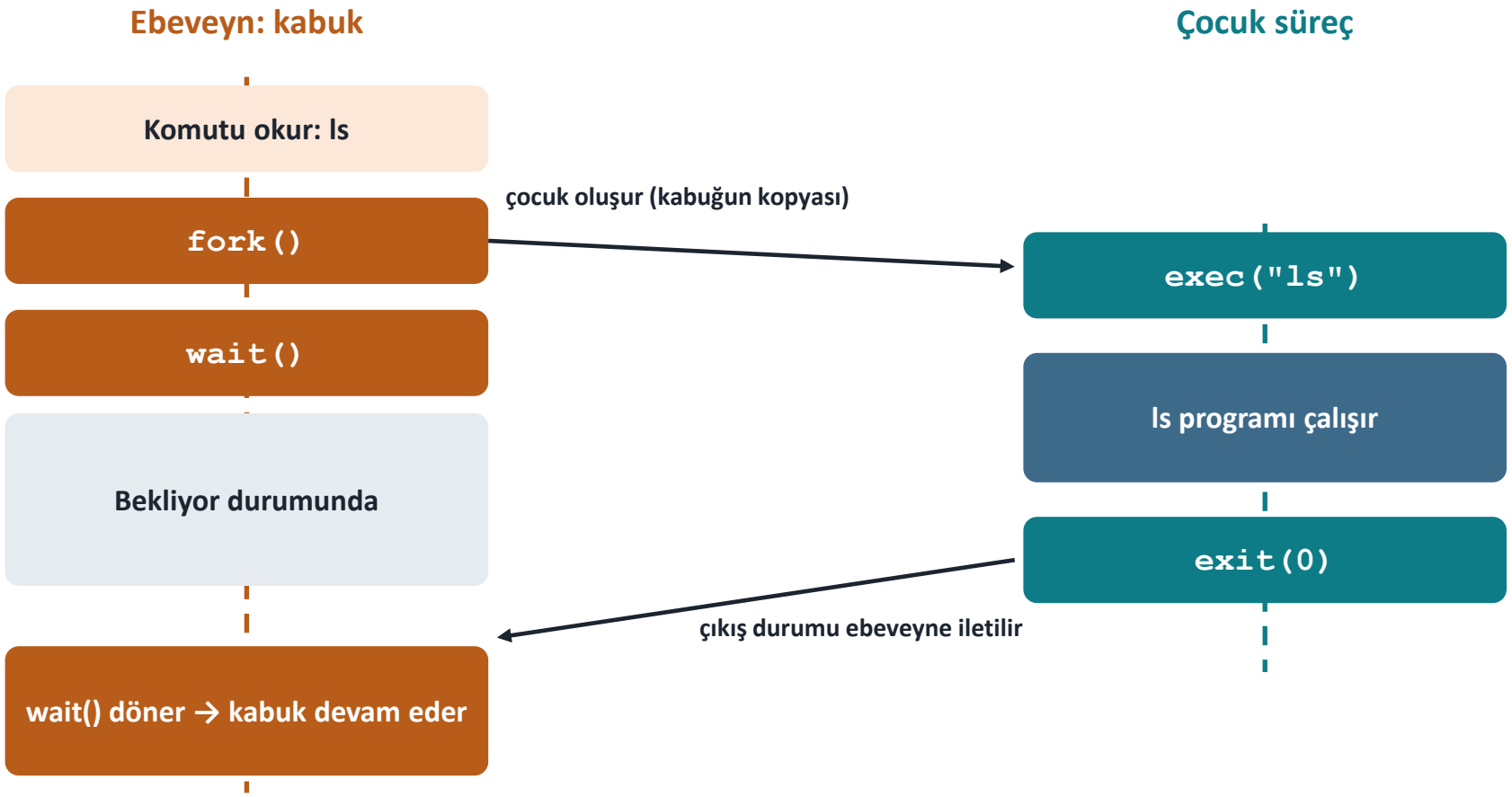
Kaynak: Silberschatz vd., Bölüm 3.3.1; Tanenbaum & Bos, Bölüm 1.6.2 · Açık dosyaların korunması UNIX davranışıdır

fork() ile exec() Aynı Şey Değildir

Ölçüt	fork()	exec()
Yeni süreç oluşturur mu?	Evet: çocuk süreç	Hayır
PID	Çocuğun PID'i farklıdır	Aynı PID korunur
Çalışan program	Çocuk, ebeveynle aynı programı sürdürür	Süreç yeni bir programa geçer
Dönüş	İki kez döner: ebeveynde ve çocukta	Başarılıysa geri dönmez
Tipik kullanım	Eşzamanlı yeni bir yürütme dizisi başlatmak	Bir sürecin başka bir programı çalıştırması

Kabuk bir komutu çalıştırırken ikisini birlikte kullanır: önce fork() ile çocuk oluşturur, sonra çocuk exec() ile istenen programa dönüşür.

Kabuk Bir Komutu Nasıl Çalıştırır? fork() + exec() + wait()



Neden böyle?

- Kabuk kendi görüntüsünü korur; işi çocuğa yaptırır
- Komut sonunda '&' verilirse kabuk wait() çağırmadan hemen yeni komut alır (arka plan)

Silberschatz vd., Bölüm 3.3.1 ve Şekil 3.10 · Hafta 2'deki "kabuk bir sistem programıdır" fikrinin devamı

Kod: fork(), exec() ve wait() Birlikte

```
pid_t pid = fork();

if (pid < 0) {                  /* hata */
    fprintf(stderr, "fork başarısız\n");
    return 1;
} else if (pid == 0) {          /* çocuk */
    execlp("/bin/ls", "ls", NULL);
} else {                        /* ebeveyn */
    wait(NULL);
    printf("Çocuk tamamlandı\n");
}
```

Çocuk dalı

execlp ile süreç görüntüsü ls programına dönüşür; ls bitince çocuk sonlanır

Ebeveyn dalı

wait(NULL) ile çocuk bitene kadar bekler, sonra mesajı yazar

Sıra

“Çocuk tamamlandı” her zaman ls çıktısından sonra yazılır — wait() bunu garanti eder

Silberschatz vd., Şekil 3.9 temel alınarak sadeleştirilmiştir

Gerçek Sistem Örneği: Linux'ta Süreçleri Gözlemlemek

```
$ ps -el
F S   UID   PID  PPID ... CMD
4 S     0     1     0 ... init
0 S  1000  2401  2398 ... bash
0 R  1000  2915  2401 ... ps
```

Örnek çıktı sadeleştirilmiştir; değerler temsilîdir

`PID`	sürecin kimliği
`PPID`	ebeveynin kimliği: ps'nin ebeveyni bash
`S`	durum: R çalışıyor/hazır, S uyuyor (bekliyor)
`getpid() / getppid()`	program içinden kendi ve ebeveyn PID'ini öğrenme

Gözlem: ps'yi kabuktan çalıştırdığınızda kabuk fork() + exec() yapar; bu yüzden ps'nin PPID'i kabuğun PID'idir.

Kaynak: Silberschatz vd., Bölüm 3.3.1 (ps komutu)

Süreç Sonlandırma (Process Termination)

Normal çıkış

GÖNÜLLÜ

İş bitti: exit() ya da main'den return

Hata ile çıkış

GÖNÜLLÜ

Program bir hata fark eder, kendisi sonlanır (ör. dosya bulunamadı)

Ölümcül hata

GÖNÜLSÜZ

Geçersiz komut, geçersiz bellek erişimi, sıfıra bölme → tuzak

Başka süreç sonlandırır

GÖNÜLSÜZ

Genellikle yalnızca ebeveyn ya da yetkili süreç yapabilir

exit() ve wait()

Süreç exit(durum) ile sonlanır; İS kaynaklarını geri alır. Ebeveyn wait(&durum) ile çıkış durumunu ve biten çocuğun PID'ini alır.

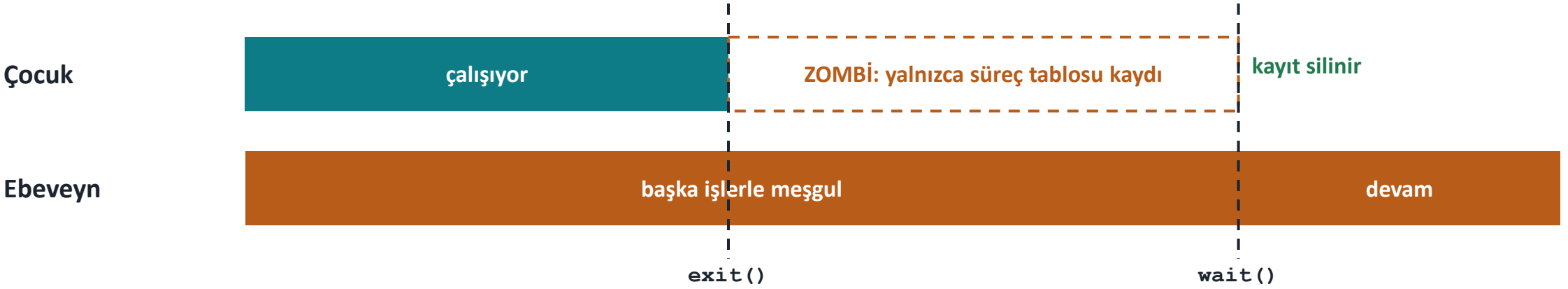
Ebeveyn çocuğu ne zaman sonlandırır?

- Çocuk kaynak sınırını aştıysa ya da görevine gerek kalmadıysa
- Ebeveyn sonlanıyorsa ve İS çocuğun sürmesine izin vermiyorsa (ardışık sonlandırma)

Kaynaklar: Tanenbaum & Bos, Bölüm 2.1.3 (dört durum); Silberschatz vd., Bölüm 3.3.2

Zombi Süreç (Zombie Process)

Zombi: Sonlanmış ama ebeveyni henüz wait() çağırıp çıkış durumunu almamış süreç.



Gerçek

Kaynakları geri alınmıştır; CPU'da çalışmaz. Süreç tablosunda PID ve çıkış durumu kalır. Her süreç bu duruma kısa süreliğine uğrar.

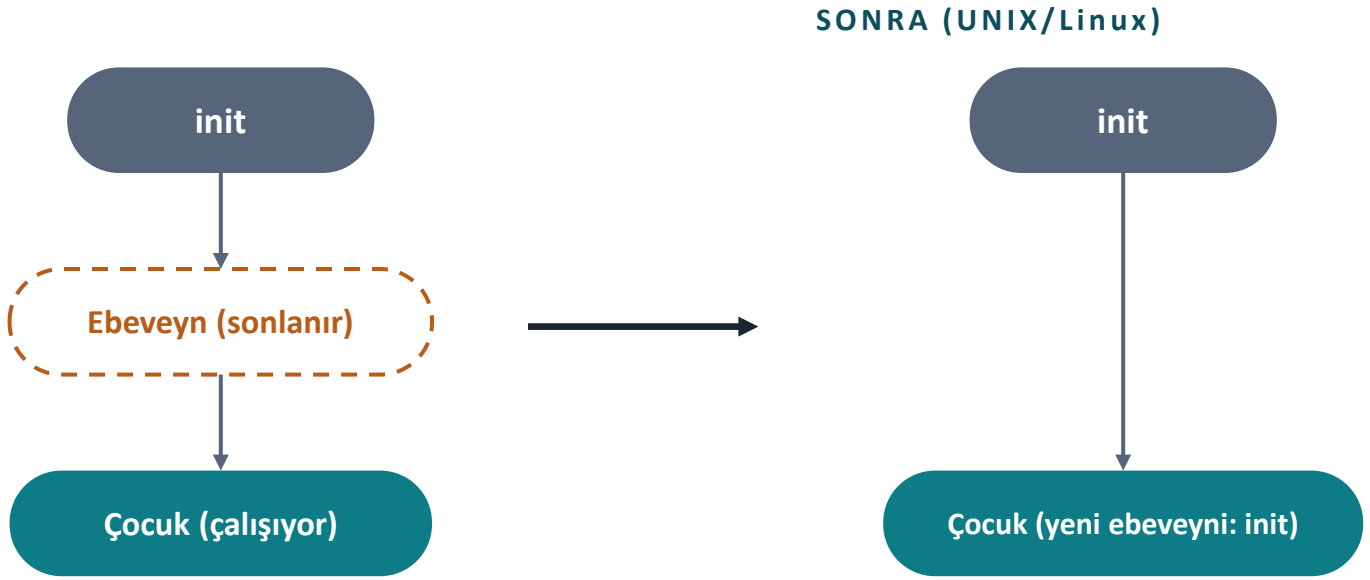
Yanlış anlama

“Zombi, hâlâ çalışan ölü bir programdır” — HAYIR. Zombi hiçbir kod yürütmez; yalnızca bir tablo girdisidir.

Kaynak: Silberschatz vd., Bölüm 3.3.2

Yetim Süreç (Orphan Process)

Yetim: Ebeveyni, wait() çağırmadan çocuğundan önce sonlanmış ve hâlâ çalışan süreç.



- UNIX ve Linux, yetim süreçlere yeni ebeveyn olarak init sürecini atar
- init düzenli olarak wait() çağırır; böylece yetim sonlandığında çıkış durumu toplanır ve kaydı silinir
- Yetim, zombiden farklı olarak hâlâ çalışıyor olabilir

Kitap temelli kavramsal açıklama: güncel sistemlerde yeniden ebeveynleme ayrıntıları farklı olabilir. Silberschatz vd., Bölüm 3.3.2

Zombi ile Yetim Karşılaştırması

Ölçüt	Zombi süreç	Yetim süreç
Süreç çalışıyor mu?	Hayır, sonlanmıştır	Evet, çalışmayı sürdürebilir
Ebeveyn durumu	Yaşıyor ama henüz wait() çağırmadı	Çocuktan önce sonlandı
Ne kalır?	Süreç tablosunda PID ve çıkış durumu	Tam bir süreç: belleği, kaynakları
Nasıl çözülür?	Ebeveyn wait() çağırır	init (yeni ebeveyn) üstlenir, sonunda wait() çağırır
Olası sorun	Çok sayıda birikirse süreç tablosu ve PID'ler tükenebilir	Genellikle sorun değildir

Aynı süreç ikisi birden de olabilir: yetim kalan bir çocuk sonlandığında, yeni ebeveyni wait() çağırana kadar kısa süre zombi olur.

Süreçler Tamamen Bağımsız mı Çalışır?

Bağımsız süreç

Başka süreçleri etkilemez ve onlardan etkilenmez; veri paylaşmaz.

İşbirliği yapan süreç (cooperating process)

Başka süreçleri etkileyebilir ya da onlardan etkilenebilir; veri paylaşır.

Neden işbirliği? Dört gerekçe

1 Bilgi paylaşımı

Aynı veriye birden çok kullanıcı ya da uygulama erişmek ister (ör. ortak bir dosya)

2 Hesaplama hızlanması

Bir iş alt görevlere bölünüp paralel çalıştırılır (birden çok işlem birimi gerekir)

3 Modülerlik

Sistem ayrı süreçlere bölünmüş işlevlerle kurulur

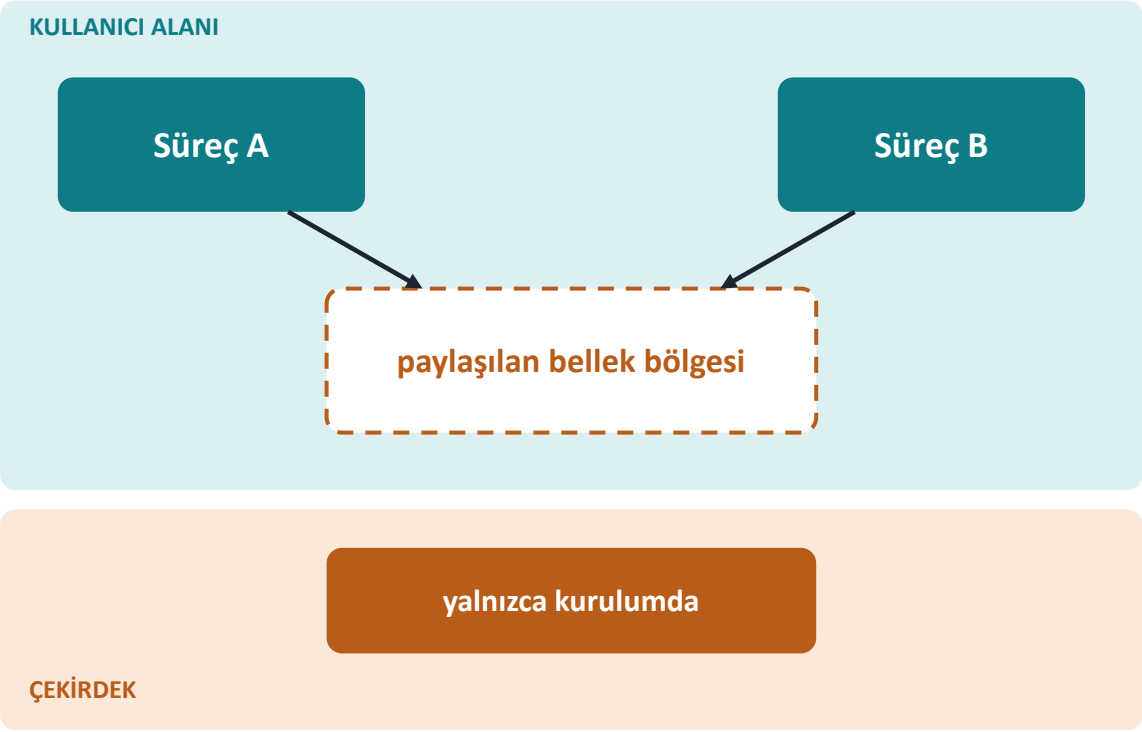
4 Kolaylık

Kullanıcı aynı anda birden çok işi yürütür (düzenleme, derleme, dinleme)

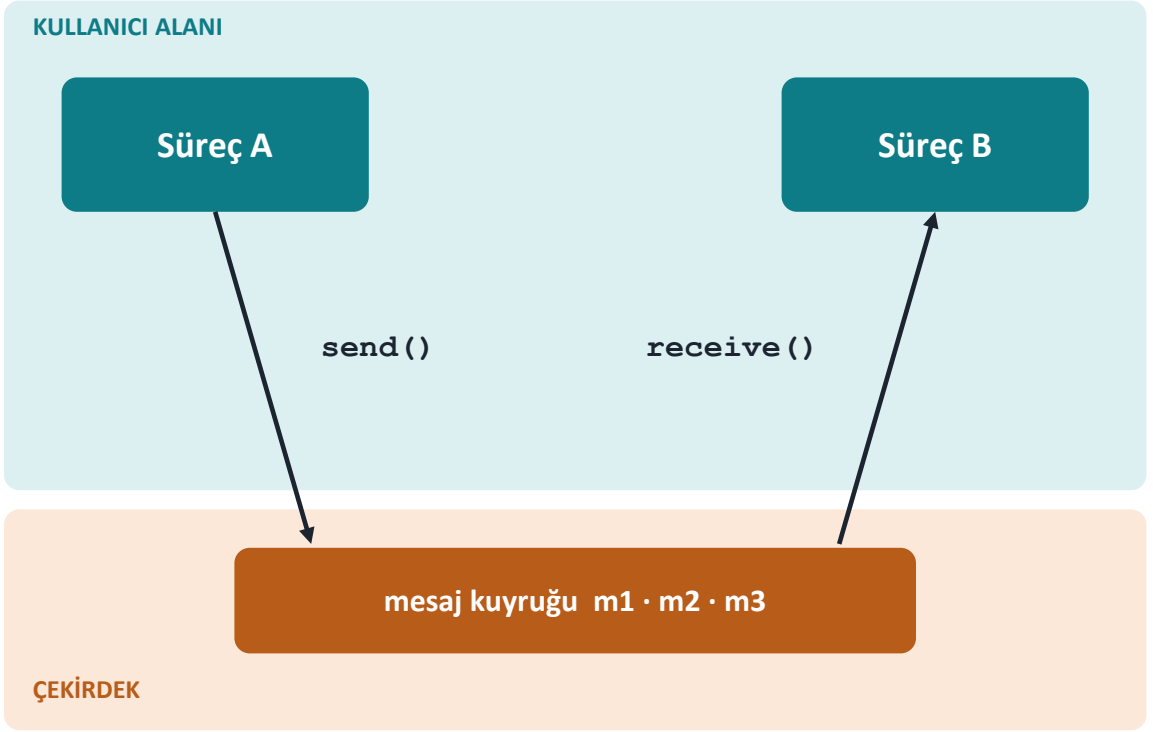
Kaynak: Silberschatz vd., Bölüm 3.4 · İşbirliği bir IPC (Interprocess Communication) mekanizması gerektirir

İki Temel IPC Modeli

(a) Paylaşımlı bellek (Shared Memory)



(b) Mesaj geçirme (Message Passing)



Paylaşımlı bellekte çekirdek yalnızca bölgeyi kurarken devrededir; mesaj geçirmede her mesaj çekirdek üzerinden iletilir.

Silberschatz vd., Şekil 3.12

Paylaşımlı Bellek (Shared Memory)

- İletişim kuran süreçler ortak bir bellek bölgesi oluşturur
- Normalde İS bir sürecin başka bir sürecin belleğine erişmesini engeller; paylaşımlı bellek için süreçler bu kısıtın kaldırılmasında anlaşır
- Bölge kurulduktan sonra erişim sıradan bellek erişimidir; çekirdek araya girmez
- Verinin biçimi ve konumu süreçlerce belirlenir, İS denetiminde değildir

Olası kazanım

Aynı makinede bellek hızında iletişim: yüksek başarım potansiyeli

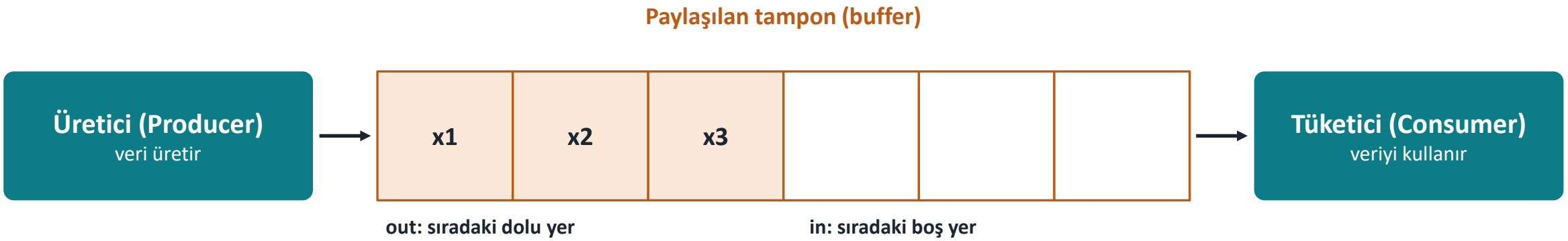
Yeni sorun

İki süreç aynı konuma aynı anda yazarsa? Erişimlerin eşgüdümü süreçlerin sorumluluğundadır.

İleri bağlantı: Eşzamanlı erişimin doğurduğu sorunlar ve çözümleri Hafta 6–7'de, Senkronizasyon konusunda ele alınacaktır.

Kaynak: Silberschatz vd., Bölüm 3.4.1

Üretici–Tüketici Problemine Ön Bakış



Sınırsız tampon (unbounded)

Tampon boyutu pratikte sınırsızdır: tüketici yeni öğe için bekleyebilir, üretici her zaman üretebilir.

Sınırlı tampon (bounded)

Tampon sabit boyutludur: tampon boşsa tüketici, doluysa üretici beklemelidir.

Bu hafta yalnızca paylaşımlı bellek ile IPC örneği olarak tanıtılır. Doğru eşgüdüm (senkronizasyon) Hafta 6'nın konusudur.

Mesaj Geçirme (Message Passing)




```
send(mesaj)
receive(mesaj)
```

- Paylaşılan adres uzayı olmadan iletişim ve eşgüdüm
- Mesajlar sabit ya da değişken boyutlu olabilir
- Önce iletişim bağlantısı kurulur; tasarım seçenekleri: doğrudan/dolaylı, eşzamanlı/eşzamansız, tamponlama

Özellikle kullanışlı olduğu yerler

- Küçük miktarda veri alışverişi
- Ağla bağlı farklı bilgisayarlardaki süreçler (dağıtık sistemler)
- Adres uzaylarının korunmasının istendiği durumlar

Kaynaklar: Silberschatz vd., Bölüm 3.4.2; Tanenbaum & Bos, Bölüm 2.3.8

Paylaşımlı Bellek ile Mesaj Geçirme Karşılaştırması

Ölçüt	Paylaşımlı bellek	Mesaj geçirme
Çekirdek ne zaman devrede?	Yalnızca bölge kurulurken	Genellikle her mesajda (sistem çağrısı)
Veri miktarı	Büyük ve sık veri paylaşımına uygun	Küçük veri alışverişine uygun
Eşgüdüm sorumluluğu	Süreçlerde (senkronizasyon gerekir)	Büyük ölçüde mesaj mekanizmasında
Farklı bilgisayarlar arası	Doğrudan uygun değil	Uygun; gerçekleştirmesi daha kolay
Başarım	Genellikle hızlı olabilir	Çok çekirdekli sistemlerde bazı araştırmalara göre daha iyi olabilir

Mutlak bir kazanan yoktur: başarım gerçekleştirmeye ve donanıma bağlıdır. Çoğu sistem iki modeli de sunar.

Kaynak: Silberschatz vd., Bölüm 3.4 (önbellek tutarlılığı nedeniyle çok çekirdekli sistemlerdeki gözlem dahil)

Doğrudan ve Dolaylı İletişim

Doğrudan iletişim (direct)




```
send(Q, mesaj)
receive(P, mesaj)
```

- Süreçler birbirini açıkça adlandırır
- Bağlantı otomatik kurulur; tam olarak iki süreç arasında
- Sınırlama: bir sürecin adı değişirse onu kullanan tüm kodlar etkilenir

Dolaylı iletişim (indirect)




```
send(A, mesaj)
receive(A, mesaj)
```

- Mesajlar posta kutusuna (port) gönderilir ve oradan alınır
- Ortak posta kutusu paylaşan süreçler arasında bağlantı vardır
- Bir bağlantı ikiden fazla süreci içerebilir

Kaynak: Silberschatz vd., Bölüm 3.4.2.1

Engelleyen ve Engellemeyen Mesaj Geçirme

İşlem	Engelleyen (blocking / eşzamanlı)	Engellemeyen (nonblocking / eşzamansız)
send()	Gönderen, mesaj alıcıya ya da posta kutusuna ulaşana kadar bekler	Gönderen mesajı bırakır ve hemen devam eder
receive()	Alıcı, bir mesaj gelene kadar bekler	Alıcı geçerli bir mesaj ya da boş (null) değer alır ve devam eder

Durum diyagramıyla bağlantı

Engelleyen receive() çağıran ve mesaj bulamayan süreç Çalışıyor → Bekliyor durumuna geçer; mesaj gelince Hazır olur.

Buluşma (rendezvous)

send() ve receive() ikisi de engelleyiciyse gönderen ile alıcı buluşur. Üretici–tüketici, engelleyen çağrılarla mesaj geçirmede kolayca çözülür.

Kaynak: Silberschatz vd., Bölüm 3.4.2.2

Tamponlama: Mesaj Kuyruğunun Kapasitesi

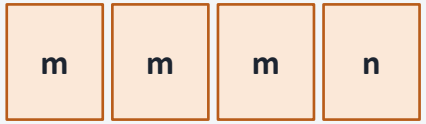
Sıfır kapasite



Kuyrukta bekleyen mesaj olamaz

Gönderen, alıcı mesajı alana kadar engellenir

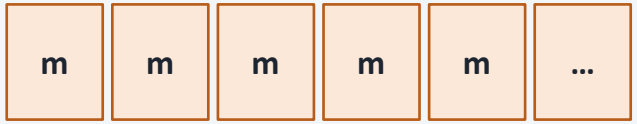
Sınırlı kapasite



En fazla n mesaj bekleyebilir

Kuyruk doluysa gönderen yer açılana kadar engellenir

Sınırsız kapasite



Kuyruk uzunluğu kuramsal olarak sınırsız

Gönderen hiç engellenmez

Sıfır kapasite "tamponsuz" sistem; diğer ikisi "otomatik tamponlamalı" sistem olarak adlandırılır.

Kaynak: Silberschatz vd., Bölüm 3.4.2.3

Gerçek Sistem Örneği: POSIX Paylaşımlı Bellek

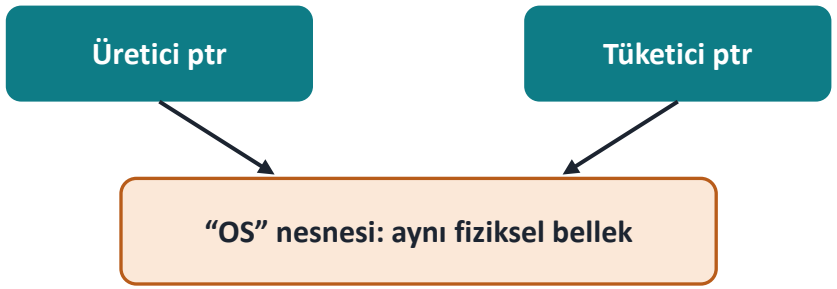
```
/* Üretici */
fd  = shm_open("OS", O_CREAT | O_RDWR, 0666);
ftruncate(fd, 4096);
ptr = mmap(0, 4096, PROT_WRITE, MAP_SHARED, fd, 0);
sprintf(ptr, "Merhaba");

/* Tüketici */
fd  = shm_open("OS", O_RDONLY, 0666);
ptr = mmap(0, 4096, PROT_READ, MAP_SHARED, fd, 0);
printf("%s", ptr);
```

`shm_open()` adlandırılmış paylaşımlı bellek nesnesi oluşturur ya da açar

`ftruncate()` nesnenin boyutunu ayarlar

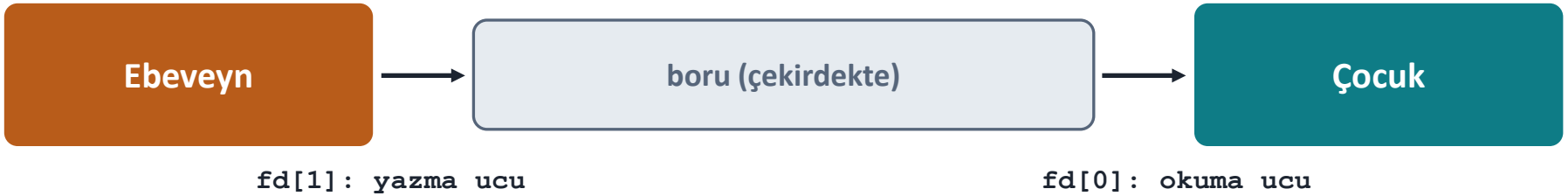
`mmap()` nesneyi sürecin adres uzayına eşler



İki süreç farklı adreslerde (ptr) olsa da aynı fiziksel veriyi görür: İS, iki sürecin sayfalarını aynı fiziksel belleğe eşler (eşleme ayrıntısı Hafta 9–10).

Silberschatz vd., Bölüm 3.5.1 (Şekil 3.17–3.18) sadeleştirilmiştir; hata denetimi ve shm_unlink() çıkarılmıştır

Sıradan Boru (Ordinary Pipe)




```
int fd[2];
pipe(fd);                /* boruyu oluştur */
if (fork() > 0) {        /* ebeveyn */
    close(fd[0]);  write(fd[1], "veri", 5);
} else {                 /* çocuk */
    close(fd[1]);  read(fd[0], buf, 5);
}
```

- Üretici–tüketici tarzında iletişim: bir uçtan yazılır, diğer uçtan okunur
- Tek yönlüdür; iki yönlü iletişim için iki boru gerekir
- Oluşturan süreç dışından erişilemez: genellikle ebeveyn–çocuk arasında kullanılır (çocuk açık uçları fork ile devralır)
- İletişim bittiğinde ve süreçler sonlandığında boru ortadan kalkar

Kaynak: Silberschatz vd., Bölüm 3.6.3.1 · Windows'taki karşılığı: anonim borular

Adlandırılmış Boru (Named Pipe) ile Karşılaştırma

Ölçüt	Sıradan boru	Adlandırılmış boru (UNIX: FIFO)
Kimler kullanabilir?	Ebeveyn–çocuk gibi ilişkili süreçler	İlişkisiz süreçler de
Ömür	İletişim bitince ortadan kalkar	Süreçler bittikten sonra da var olmayı sürdürür
Ad	Adı yoktur	Dosya sisteminde bir ada sahiptir
Oluşturma (UNIX)	pipe()	mkfifo()
Yön	Tek yönlü	UNIX FIFO: iki yönlü olabilir ama yarı çift yönlü; iki yönde veri için genellikle iki FIFO kullanılır

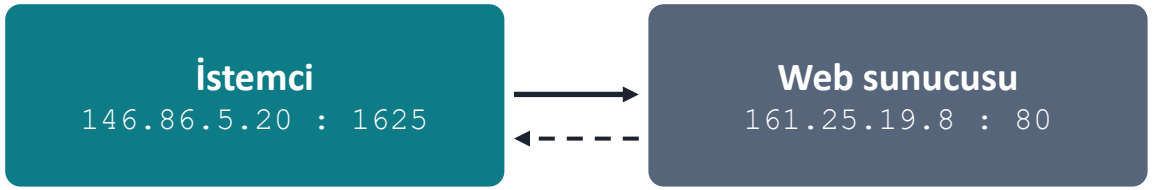
Ayrıntılı davranış işletim sistemine göre değişir (ör. Windows adlandırılmış boruları); bu derste temel fark yeterlidir.

Kaynak: Silberschatz vd., Bölüm 3.6.3.2

Soket ve Uzak Yordam Çağrısı (RPC): Kısa Bakış

Soket (Socket)

İletişimin bir uç noktası: IP adresi + port numarası



- Bir bağlantı bir soket çiftiyle tanımlanır
- Sunucular iyi bilinen portlarda dinler (ör. HTTP 80)
- Aynı makinede de kullanılabilir: 127.0.0.1 (geri döngü)

Uzak Yordam Çağrısı (RPC)

Uzaktaki bir yordamı yerel bir yordam gibi çağırmayı amaçlayan soyutlama

İstemci programı

İstemci koçanı (stub): parametreleri paketler

ağ üzerinden mesaj

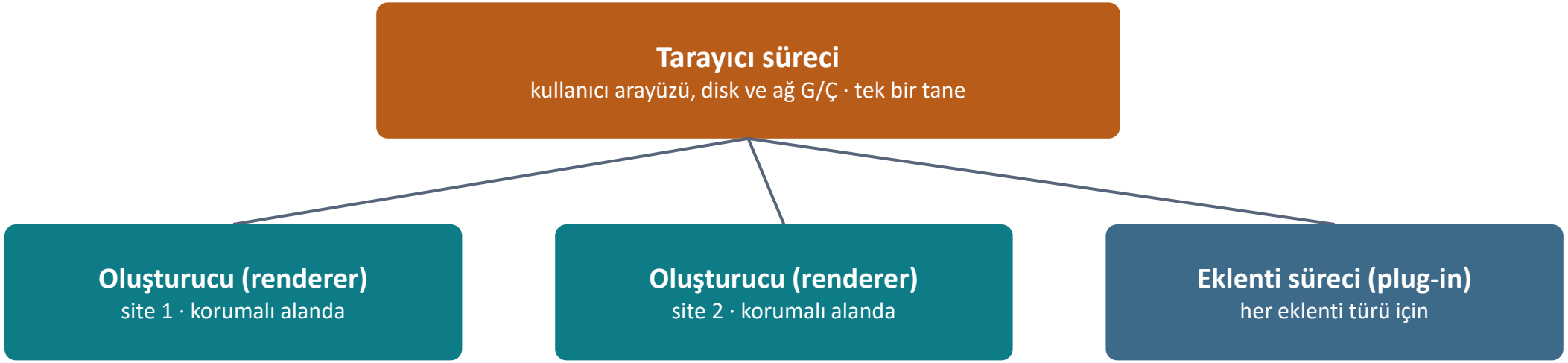
Sunucu koçanı: açar, yordamı çağırır

Bu hafta yalnızca IPC bakış açısıyla tanıtılır; ağ programlama bu dersin konusu değildir.

Kaynak: Silberschatz vd., Bölüm 3.6.1–3.6.2 (adres örnekleri kitaptandır)

Kitap Temelli Örnek: Chrome'un Çok Süreçli Mimarisi

Sorun: Tüm sekmeler tek süreçteyse, bir sekmedeki web uygulamasının çökmesi tüm tarayıcıyı çökertir.



Yalıtım
Siteler birbirinden ayrı süreçlerde çalışır

Kararlılık
Bir site çökerse yalnızca onun oluşturucu süreci etkilenir

Güvenlik
Oluşturucular disk ve ağ erişimi kısıtlı korumalı alanda (sandbox) çalışır

Kitap temelli örnek (Silberschatz vd., Bölüm 3.4); güncel Chrome'un iç mimarisi farklılık gösterebilir. Süreçler birbiriyle IPC ile iletişir.

Bu Hafta Neler Öğrendik?

1. Süreç, çalışmakta olan programdır; program pasif, süreç etkin varlıktır
2. Süreç beş durumdan geçer; geçişleri olaylar tetikler
3. İS süreci PCB ile izler; bağlam değiştirme bir ek yüktür
4. fork() yeni süreç oluşturur; exec() görüntüyü değiştirir; wait() bekler
5. Zombi: bitmiş ama toplanmamış · Yetim: ebeveyni bitmiş
6. IPC: paylaşımlı bellek ya da mesaj geçirme; borular, soketler, RPC

Gelecek hafta: Bir süreç içinde birden çok yürütme akışı olabilir mi? → İş parçacıkları ve çok çekirdekli programlama

Yaygın Yanlış Anlamalar

YANLIŞ	DOĞRU
“Program = süreç”	Program pasif dosyadır; süreç onun çalışan örneğidir
“Bir program yalnızca bir süreç oluşturabilir”	Aynı programdan birçok süreç çalışabilir
“Bağlam değiştirme yararlı iş yapar”	Saf ek yüktür; o sırada kullanıcı işi yapılmaz
“fork() yeni bir program yükler”	fork() yeni süreç oluşturur; çocuk aynı programı sürdürür
“exec() yeni süreç oluşturur”	exec() mevcut sürecin görüntüsünü değiştirir; PID aynıdır
“Zombi süreç hâlâ CPU'da çalışır”	Zombi kod yürütmez; yalnızca süreç tablosu kaydıdır
“Yetim ile zombi aynıdır”	Yetimin ebeveyni bitmiştir ve kendisi çalışabilir; zombi ise bitmiştir
“IPC yalnızca ağ üzerinden yapılır”	Paylaşımlı bellek ve borular aynı makinede IPC sağlar

Kontrol Soruları

KAVRAMSAL · DURUM SENARYOSU

S1 **Çalışıyor durumundaki bir süreç diskten okuma isteği yaptığında hangi duruma geçer? Okuma bittiğinde neden doğrudan Çalışıyor durumuna geçmez?**

İlgili bölüm: 2 · Süreç durumları

S2 **exec() neden yeni bir PID oluşturmak zorunda değildir?**

İlgili bölüm: 5 · Süreç oluşturma

Kontrol Soruları

fork() KOD ANALİZİ

S3

fork(); fork(); kodundan sonra kaç süreç vardır? Süreç ağacını çiziniz.

Her sürecin hangi fork çağrılarını yürüttüğünü izleyin.

S4

for (int i = 0; i < 3; i++) fork(); döngüsü sonunda kaç süreç vardır? printf("X") döngüden sonra kaç kez yazdırılır?

İpucu: Döngüyü üç ayrı fork() satırı gibi düşünün.

Kontrol Soruları

PCB · BAĞLAM DEĞİŞTİRME

S5 **Bağlam değiştirme sırasında PCB'ye hangi bilgiler kaydedilmelidir? Program sayacı kaydedilmeseydi ne olurdu?**

İlgili bölüm: 3 · PCB ve bağlam değiştirme

S6 **Bağlam değiştirme neden “ek yük” olarak adlandırılır? Çok sık bağlam değiştirmenin sistem verimine etkisi ne olur?**

İlgili bölüm: 3 · Bağlam değiştirme

Kontrol Soruları

IPC · KISA SENARYO

S7 **İki süreç aynı makinede çok büyük miktarda veriyi sık sık paylaşacaksa paylaşımlı bellek neden uygun olabilir? Hangi yeni sorun ortaya çıkar?**

İlgili bölüm: 7 · Paylaşımlı bellek

S8 **Bir ebeveyn çocuklarını hiç wait() ile beklemeden uzun süre çalışmayı sürdürürse sistemde ne birikir? Ebeveyn sonlanırsa ne olur?**

İlgili bölüm: 6 · Zombi ve yetim süreçler

Kaynaklar

ANA KAYNAK

Silberschatz, A., Galvin, P. B., & Gagne, G. (2013). *Operating System Concepts* (9th ed.). John Wiley & Sons.

Bu hafta: Bölüm 3 — 3.1–3.6

DESTEKLEYİCİ KAYNAK

Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.

Bu hafta: Bölüm 2 — 2.1 (2.1.1–2.1.6), 2.3.8 · Bölüm 1.6.2

Not: Bu sunumdaki diyagramlar, belirtilen yerlerde kaynak kitaplardaki şekiller temel alınarak yeniden çizilmiştir. PID ve çıktı değerleri temsilîdir.